



ÉCOLE NATIONALE
DES SCIENCES
APPLIQUÉES D'OUJDA

UNIVERSITÉ MOHAMMED
PREMIER



ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES D'OUJDA

Filière : Génie des Systèmes Informatiques et Cybersécurité

GSICS 4 — Année universitaire 2025 / 2026

PROJET DE FIN D'ANNÉE (PFA)

Détection d'attaques par indicateurs
de compromission (IoC)

Apprentissage fédéré décentralisé sur Blockchain
avec protection contre le reverse engineering

Réalisé par

Abiza Fadwa

El Aouni Ghizlane

El Kadiri Boutchich Radia

Encadrants

Prof. Mohammed Ouadoud

Mr. Oussama Bouzerda

Remerciements

Ce projet est l'aboutissement d'un travail collectif que nous avons mené avec passion et détermination. Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation.

Nous remercions tout d'abord notre encadrant, Professeur Mohammed Ouadoud, pour sa disponibilité, ses conseils avisés et son accompagnement tout au long de ce projet. Sa rigueur et sa bienveillance ont été pour nous une source d'inspiration et de motivation.

Nous adressons également nos sincères remerciements à Monsieur Oussama Bouzerda pour son suivi régulier, ses retours constructifs et son aide précieuse dans la résolution des difficultés techniques rencontrées.

Nous remercions chaleureusement l'ensemble des enseignants de la filière Génie des Systèmes Informatiques et Cybersécurité pour la qualité de la formation dispensée et leur disponibilité.

Que toutes ces personnes trouvent ici l'expression de notre gratitude la plus sincère.

Résumé

Ce projet de fin d'année porte sur la conception d'un système de détection d'attaques réseau basé sur les indicateurs de compromission (IoC), combinant apprentissage fédéré décentralisé et blockchain Hyperledger Fabric. L'architecture proposée repose sur quatre clients qui entraînent localement un modèle CNN-LSTM bidirectionnel sur le jeu de données CIC-IDS2018, sans recourir à un serveur central : l'agrégation des poids (FedAvg) est calculée de manière pair-à-pair par chaque client. Cette suppression du serveur central élimine le point de défaillance unique et répond à une exigence forte de résilience face aux pannes ou aux attaques ciblant un nœud central.

La traçabilité immuable des échanges entre les quatre clients est assurée par un réseau Hyperledger Fabric réel (et non simulé), sur lequel un chaincode écrit en JavaScript enregistre, à chaque round, le hash SHA-256 des poids agrégés ainsi que l'identité du client émetteur. La sécurité des poids échangés est renforcée par l'étude de deux couches complémentaires : TenSEAL, qui permet une agrégation par chiffrement homomorphe CKKS (démontrée à titre de preuve de concept), et Opacus, qui injecte du bruit différentiel dans les gradients pendant l'entraînement local afin de limiter le risque de reverse engineering des données d'origine à partir des poids partagés.

Les expérimentations menées montrent qu'en l'absence de toute couche de sécurité, le modèle converge rapidement vers un F1-score parfait. L'introduction de la confidentialité différentielle avec un budget de confidentialité ϵ trop faible bloque significativement l'apprentissage, illustrant un compromis fondamental entre protection de la vie privée et performance du modèle. Ce travail démontre la faisabilité d'une architecture de détection d'IoC à la fois décentralisée, traçable et protégée contre la rétro-ingénierie des modèles partagés.

Mots-clés : Indicateurs de compromission, apprentissage fédéré décentralisé, blockchain Hyperledger Fabric, CNN-LSTM, chiffrement homomorphe, confidentialité différentielle, détection d'intrusions, CIC-IDS2018.

Abstract

This final year project focuses on designing a network attack detection system based on Indicators of Compromise (IoC), combining decentralized federated learning and Hyperledger Fabric blockchain. The proposed architecture relies on four clients that locally train a bidirectional CNN-LSTM model on the CIC-IDS2018 dataset, without relying on a central server : weight aggregation (FedAvg) is computed in a peer-to-peer fashion by each client. Removing the central server eliminates the single point of failure and addresses a strong resilience requirement against node failures or targeted attacks on a central coordinator.

The immutable traceability of exchanges between the four clients is ensured by a real (not simulated) Hyperledger Fabric network, on which a JavaScript chaincode records, at every round, the SHA-256 hash of the aggregated weights together with the identity of the submitting client. The security of the exchanged weights is reinforced by the investigation of two complementary layers : TenSEAL, which enables aggregation through CKKS homomorphic encryption (shown as a proof of concept), and Opacus, which injects differential noise into the gradients during local training to limit the risk of reverse engineering the original data from the shared weights.

Experiments show that, in the absence of any security layer, the model quickly converges to a perfect F1-score. Introducing differential privacy with a privacy budget ε that is too small significantly hinders learning, illustrating a fundamental trade-off between privacy protection and model performance. This work demonstrates the feasibility of an IoC detection architecture that is simultaneously decentralized, traceable, and protected against reverse engineering of shared models.

Keywords : Indicators of Compromise, decentralized federated learning, Hyperledger Fabric blockchain, CNN-LSTM, homomorphic encryption, differential privacy, intrusion detection, CIC-IDS2018.

Resume (arabe)

يتمحور مشروع نهاية السنة هذا حول تصميم نظام للكشف عن هجمات الشبكات يعتمد على مؤشرات الاختراق (Indicators of Compromise - IoC)، ويجمع بين التعلّم الفيدرالي اللامركزي وتقنية Hyperledger Fabric Blockchain. تعتمد البنية المقترحة على أربعة عملاء يقوم كل منهم بتدريب نموذج CNN-LSTM ثنائي الاتجاه محليًا باستخدام مجموعة بيانات CIC-IDS2018، دون الحاجة إلى خادم مركزي، حيث يتم تنفيذ عملية تجميع أوزان النموذج (FedAvg) بطريقة الند للند (Peer-to-Peer) بواسطة كل عميل. ويساهم هذا النهج في إزالة نقطة الفشل الوحيدة، مما يعزز قدرة النظام على مقاومة الأعطال والهجمات التي قد تستهدف عقدة مركزية.

تُضمن قابلية التتبع غير القابلة للتغيير لعمليات تبادل الأوزان بين العملاء الأربعة بواسطة شبكة Hyperledger Fabric حقيقية (وليس محاكاة)، حيث يقوم عقد ذكي (Chaincode) مكتوب بلغة JavaScript بتسجيل قيمة التجزئة SHA-256 للأوزان المجمعة وهوية العميل المرسل في كل جولة تدريب. كما تم تعزيز أمن الأوزان المتبادلة من خلال دراسة طبقتين تكميليتين للحماية: الأولى باستخدام TenSEAL الذي يتيح إجراء التجميع عبر التشفير المتماثل (CKKS) كإثبات لمفهوم الحل، والثانية باستخدام Opacus الذي يضيف ضوضاء الخصوصية التفاضلية إلى التدرجات أثناء التدريب المحلي للحد من مخاطر إعادة استنتاج البيانات الأصلية انطلاقًا من الأوزان المشتركة.

أظهرت النتائج التجريبية أنه في غياب أي آلية حماية، يحقق النموذج تقاربًا سريعًا مع قيمة مثالية لمقياس F1-score. في المقابل، يؤدي تطبيق الخصوصية التفاضلية باستخدام ميزانية خصوصية ϵ منخفضة جدًا إلى إعاقة عملية التعلم بشكل ملحوظ، مما يبرز المفاضلة الأساسية بين حماية الخصوصية والحفاظ على أداء النموذج. وتبرهن هذه الدراسة على إمكانية بناء بنية فعالة للكشف عن مؤشرات الاختراق، تجمع بين اللامركزية، وقابلية التتبع، والحماية من هجمات الهندسة العكسية على النماذج المتبادلة.

الكلمات المفتاحية: مؤشرات الاختراق، التعلّم الفيدرالي اللامركزي، سلسلة الكتل Hyperledger Fabric، CNN-LSTM، التشفير المتماثل، الخصوصية التفاضلية، كشف التسلل، CIC-IDS2018.

Resume (tfinagh)

• **Overview**: I have worked on various projects in the field of machine learning and blockchain. My research interests include Federated Learning, Hyperledger Fabric Blockchain, and Intrusion Detection. I have published several papers in the field of machine learning and blockchain, and I have been involved in several projects related to these topics.

• **Research Interests**: Federated Learning, Hyperledger Fabric Blockchain, Intrusion Detection, CNN-BiLSTM, CIC-IDS2018, Homomorphic Encryption, Differential Privacy, Federated Averaging (FedAvg), peer-to-peer.

• **Publications**: I have published several papers in the field of machine learning and blockchain. My publications include: "A Secure and Efficient Federated Learning Framework for Intrusion Detection" (2020), "A Secure and Efficient Federated Learning Framework for Intrusion Detection" (2020), "A Secure and Efficient Federated Learning Framework for Intrusion Detection" (2020), "A Secure and Efficient Federated Learning Framework for Intrusion Detection" (2020), "A Secure and Efficient Federated Learning Framework for Intrusion Detection" (2020).

• **Projects**: I have been involved in several projects related to machine learning and blockchain. My projects include: "A Secure and Efficient Federated Learning Framework for Intrusion Detection" (2020), "A Secure and Efficient Federated Learning Framework for Intrusion Detection" (2020), "A Secure and Efficient Federated Learning Framework for Intrusion Detection" (2020), "A Secure and Efficient Federated Learning Framework for Intrusion Detection" (2020), "A Secure and Efficient Federated Learning Framework for Intrusion Detection" (2020).

• **Skills**: I have a strong background in machine learning and blockchain. My skills include: Federated Learning, Hyperledger Fabric, CNN-LSTM, Homomorphic Encryption, Differential Privacy, Intrusion Detection, CIC-IDS2018.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Résumé (arabe)	iv
Résumé (tifinagh)	v
Liste des abréviations	xii
Introduction générale	1
1 Contexte général	3
Introduction	3
1.1 Contexte du projet	3
1.2 Problématique	3
1.3 Objectifs du projet	4
1.4 Contributions attendues	5
Conclusion	5
2 État de l’art et fondements théoriques	6
Introduction	6
2.1 Indicateurs de compromission et détection d’intrusions	6
2.1.1 Définition des IoC	6
2.1.2 Rôle des IoC dans l’identification d’activités malveillantes	6
2.1.3 Détection par signatures, anomalies et apprentissage automatique	6
2.2 Deep learning pour la détection d’attaques réseau	7
2.2.1 Intérêt des architectures CNN	7
2.2.2 Apport des LSTM et BiLSTM	7
2.2.3 Architecture hybride retenue	7
2.3 Apprentissage fédéré	7
2.3.1 Principe général	7
2.3.2 Algorithme FedAvg	8

2.3.3	Partitionnement IID et non-IID	8
2.4	Blockchain et traçabilité décentralisée	8
2.4.1	Intérêt d'une blockchain privée pour la traçabilité	8
2.4.2	Hyperledger Fabric	9
2.4.3	Rôle du chaincode dans l'enregistrement des échanges	9
2.5	Protection contre le reverse engineering	9
2.5.1	Le risque de rétro-ingénierie des modèles fédérés	9
2.5.2	Chiffrement homomorphe avec TenSEAL	10
2.5.3	Confidentialité différentielle avec Opacus	10
2.5.4	Le compromis confidentialité-performance	10
	Conclusion	10
3	Analyse fonctionnelle	12
	Introduction	12
3.1	Expression des besoins	12
3.1.1	Besoins fonctionnels	12
3.1.2	Besoins non fonctionnels	12
3.2	Cas d'usage	13
3.2.1	Acteurs du système	13
3.2.2	Cas d'usage principaux	13
3.3	Décomposition fonctionnelle (méthode FAST)	14
3.3.1	Fonction globale	14
3.3.2	Fonctions principales et fonctions techniques	14
	Conclusion	14
4	Charte du projet	15
	Introduction	15
4.1	Matrice des risques	15
4.2	Matrice des risques	15
4.3	Planning prévisionnel et réalisé	16
4.4	Répartition des responsabilités	17
	Conclusion	17
5	Conception et implémentation	19
	Introduction	19
5.1	Vue d'ensemble de l'architecture	19
5.2	Évolutions par rapport au plan initial	20
5.3	Évolutions techniques	20
5.4	Architecture du modèle CNN-LSTM	21
5.4.1	Flux des données dans le modèle	21

5.4.2	Compatibilité avec la confidentialité différentielle	22
5.4.3	Validation de l'architecture	22
5.5	Préparation et partitionnement des données	22
5.5.1	Jeu de données CIC-IDS2018	22
5.5.2	Partitionnement entre les quatre clients	22
5.6	Apprentissage fédéré décentralisé pair-à-pair	23
5.6.1	Suppression du serveur central	23
5.6.2	Extrait d'implémentation	23
5.7	Déploiement du réseau Hyperledger Fabric	24
5.7.1	Choix d'architecture réseau	24
5.7.2	Composants du réseau déployé	24
5.8	Chaincode de traçabilité	25
5.8.1	Fonctions exposées	25
5.8.2	Extrait du chaincode	26
5.8.3	Calcul du hash des poids agrégés	26
5.8.4	Pont entre le pipeline Python et le réseau Fabric	27
5.9	Intégration des couches de sécurité	27
5.9.1	Chiffrement homomorphe avec TenSEAL	27
5.9.2	Confidentialité différentielle avec Opacus	28
	Conclusion	28
6	Résultats et interprétation	29
	Introduction	29
6.1	Métriques d'évaluation	29
6.2	Scénarios expérimentaux	29
6.3	Scénario 1 – Sans aucune sécurité (<code>main_5rounds.py</code>)	30
6.4	Scénario 2 – Chiffrement homomorphe TenSEAL (démonstration) (<code>main_tenseal_only.py</code>)	30
6.5	Scénario 3 – Opacus avec budget de confidentialité strict ($\varepsilon \approx 0,49$) (<code>main_opacus_balanced.py</code>)	32
6.6	Analyse du compromis confidentialité-performance	33
6.7	Validation de la traçabilité blockchain	33
	Conclusion	36
7	Organisation du travail, planning et livrables	37
	Introduction	37
7.1	Stack technique complète	37
7.2	Répartition effective du travail	38
7.3	Livrables produits	38
	Conclusion	38

Conclusion générale	40
A Configuration expérimentale	43
A.1 Environnement matériel et logiciel	43
A.2 Hyperparamètres d'entraînement	44
A.3 Paramètres de la couche de sécurité	44
B Détails du modèle CNN-LSTM	45
B.1 Définition complète de la classe du modèle	45
B.2 Partitionnement IID	45
B.3 Partitionnement non-IID (Dirichlet)	46
C Extraits du chaincode et commandes Fabric	47
C.1 Chaincode complet (lib/fTracker.js)	47
C.2 Commandes de déploiement du réseau Fabric	48
C.3 Commande d'interrogation du registre	48
D Captures des résultats et courbes de convergence	49

Table des figures

5.1	Architecture globale du système – apprentissage fédéré décentralisé et traçabilité blockchain	20
5.2	Containers Docker actifs du réseau Hyperledger Fabric (sortie de <code>docker ps</code>)	25
6.1	Convergence du F1-score en partitionnement IID et NON IID sans sécurité (script <code>main_5rounds.py</code>)	31
6.2	Convergence du F1-score en partitionnement IID et NON IID avec sécurité (TenSEAL)(script <code>main_tenseal_only</code>)	32
6.3	Comparaison des F1-scores IID vs non-IID avec Opacus activé (script <code>main_opacus_balanced.py</code>)	33
6.4	Aperçu de l'historique des transactions enregistrées sur le chaincode <code>fltracker</code> (requête <code>GetAllTransactions</code>)	34
6.5	Extrait des journaux d'exécution du script <code>main_opacus_balanced.py</code> montrant le F1-score, le budget de confidentialité ϵ et les transactions blockchain	35
D.1	Architecture globale du système – apprentissage fédéré décentralisé et traçabilité blockchain	49
D.2	Containers Docker actifs du réseau Hyperledger Fabric (sortie de <code>docker ps</code>)	50
D.3	Aperçu de l'historique des transactions enregistrées sur le chaincode <code>fltracker</code> (requête <code>GetAllTransactions</code>)	50
D.4	Convergence du F1-score en partitionnement IID et NON IID sans sécurité (script <code>main_5rounds.py</code>)	51
D.5	Convergence du F1-score en partitionnement IID et NON IID avec sécurité (TenSEAL)(script <code>main_tenseal_only</code>)	51
D.6	Comparaison des F1-scores IID vs non-IID avec Opacus activé (script <code>main_opacus_balanced.py</code>)	52
D.7	Extrait des journaux d'exécution du script <code>main_opacus_balanced.py</code> montrant le F1-score, le budget de confidentialité ϵ et les transactions blockchain	53

Liste des tableaux

3.1	Fonctions principales et fonctions techniques associées	14
4.1	Matrice d'analyse des risques du projet	15
4.2	Planning du projet (diagramme de Gantt simplifié)	17
5.1	Évolutions techniques par rapport au plan initial	20
6.1	Synthèse des scénarios expérimentaux et résultats obtenus	30
7.1	Pile technique complète du projet	37
A.1	Configuration de l'environnement expérimental	43
A.2	Hyperparamètres utilisés pour l'entraînement du modèle CNN-LSTM . . .	44
A.3	Paramètres TenSEAL et Opacus utilisés	44

Liste des abréviations

Abréviation	Signification
IoC	Indicator of Compromise (indicateur de compromission)
IDS	Intrusion Detection System (système de détection d'intrusions)
FL	Federated Learning (apprentissage fédéré)
P2P	Peer-to-Peer (pair-à-pair)
FedAvg	Federated Averaging (algorithme d'agrégation fédérée par moyenne)
CNN	Convolutional Neural Network (réseau de neurones convolutif)
LSTM	Long Short-Term Memory (mémoire à court et long terme)
BiLSTM	Bidirectional LSTM (LSTM bidirectionnel)
DP	Differential Privacy (confidentialité différentielle)
DPLSTM	Differentially Private LSTM (variante LSTM compatible Opacus)
CKKS	Cheon-Kim-Kim-Song (schéma de chiffrement homomorphe pour nombres flottants)
HE	Homomorphic Encryption (chiffrement homomorphe)
SHA-256	Secure Hash Algorithm 256 bits
MSP	Membership Service Provider (fournisseur de services d'adhésion Fabric)
CA	Certificate Authority (autorité de certification)
TLS	Transport Layer Security
IID	Independent and Identically Distributed (indépendant et identiquement distribué)
Non-IID	Non Independent and Identically Distributed
F1-score	Métrique harmonique de précision et de rappel
GPU	Graphics Processing Unit
CPU	Central Processing Unit
VM	Virtual Machine (machine virtuelle)
SDK	Software Development Kit

Abréviation	Signification
API	Application Programming Interface
JSON	JavaScript Object Notation

Introduction générale

Ce projet de fin d'année porte sur la conception d'un système de détection d'intrusions basé sur les indicateurs de compromission (IoC), en combinant l'intelligence artificielle, l'apprentissage fédéré décentralisé et la technologie blockchain. L'objectif est de permettre à plusieurs nœuds – ici quatre clients représentant des entités distinctes – de collaborer à l'entraînement d'un modèle de détection sans jamais centraliser les données réseau sensibles, tout en assurant la traçabilité immuable des échanges et la protection des poids du modèle contre les risques de rétro-ingénierie.

La cybersécurité moderne fait face à un double défi. D'une part, la détection d'attaques réseau nécessite des modèles d'apprentissage profond entraînés sur de grands volumes de trafic, ce qui pousse naturellement vers la centralisation des données – une centralisation qui pose des problèmes de confidentialité, de conformité réglementaire et de gouvernance lorsque les données proviennent de plusieurs organisations. D'autre part, la suppression d'un point de coordination central, bien que nécessaire pour la résilience face aux pannes ou aux attaques ciblées, soulève une question de confiance : comment garantir que les contributions de chaque participant sont honnêtes, vérifiables et historisées de façon incontestable ?

Pour répondre à ce double défi, ce projet propose une architecture qui repose sur trois piliers complémentaires. Le premier pilier est un modèle de deep learning hybride CNN-LSTM bidirectionnel, capable d'extraire des motifs locaux dans les caractéristiques du trafic réseau tout en modélisant les dépendances séquentielles. Le second pilier est un apprentissage fédéré entièrement décentralisé : chaque client entraîne le modèle sur ses propres données et procède lui-même à l'agrégation des contributions de l'ensemble des clients, sans dépendre d'un serveur central. Le troisième pilier est une blockchain privée Hyperledger Fabric, qui enregistre de manière immuable chaque échange de poids entre les clients, garantissant ainsi une traçabilité complète et vérifiable du processus d'apprentissage collaboratif.

Au-delà de la décentralisation et de la traçabilité, ce travail accorde une attention particulière à la protection des poids du modèle échangés entre les clients. En effet, des poids partagés sans protection peuvent, dans certains scénarios, être exploités pour reconstruire partiellement les données d'entraînement d'origine – un risque connu sous le nom de rétro-ingénierie ou *reverse engineering* des modèles. Deux mécanismes complémentaires

sont étudiés : le chiffrement homomorphe (TenSEAL) comme preuve de concept pour une agrégation chiffrée, et la confidentialité différentielle (Opacus) qui injecte un bruit calibré dans les gradients pour rendre statistiquement plus difficile la reconstruction des données individuelles.

Ce rapport présente, dans l'ordre, le contexte et la problématique du projet, l'état de l'art des domaines mobilisés, l'analyse fonctionnelle du système, la charte de gestion de projet, les fondements théoriques approfondis, la conception et l'implémentation technique réalisée, les résultats expérimentaux obtenus et leur interprétation, ainsi que l'organisation du travail au sein de l'équipe. Une attention particulière est portée à la transparence sur les choix techniques effectués au cours du projet, certains ayant évolué par rapport au plan initial pour des raisons de faisabilité pratique – ces évolutions sont explicitement documentées et justifiées au fil du rapport.

1. Contexte général

Introduction

Ce chapitre introduit le contexte général du projet, la problématique liée à la détection d’attaques réseau par indicateurs de compromission, et précise les objectifs concrets poursuivis par l’équipe ainsi que les contributions attendues à l’issue du travail.

1.1 Contexte du projet

Le projet s’inscrit dans le domaine de la cybersécurité et plus précisément de la détection d’intrusions réseau. Les systèmes d’information modernes génèrent des volumes considérables de trafic, dans lequel se mêlent des flux légitimes et des flux associés à des activités malveillantes : déni de service, force brute, infiltration, botnets, ou encore communications de commande et contrôle (C&C). Les indicateurs de compromission (IoC) – adresses IP suspectes, signatures de flux, motifs comportementaux anormaux – constituent une source d’information précieuse pour identifier ces activités.

Traditionnellement, l’entraînement de modèles de détection performants suppose la collecte centralisée de grandes quantités de trafic réseau, souvent issues de plusieurs organisations ou plusieurs segments d’un même système d’information. Cette centralisation entre en tension directe avec les exigences de confidentialité des données, les contraintes réglementaires, et la volonté des organisations de ne pas exposer le détail de leur trafic interne à un tiers. Par ailleurs, faire dépendre tout le système d’un point de coordination central (serveur d’agrégation) introduit un point de défaillance unique : si ce serveur tombe en panne ou est compromis, l’ensemble du processus d’apprentissage collaboratif s’arrête.

1.2 Problématique

Les systèmes classiques de détection d’intrusions nécessitent souvent la centralisation des données, ce qui soulève des problèmes de confidentialité, de scalabilité et de gouvernance. La problématique centrale de ce projet peut donc être formulée ainsi :

Comment détecter efficacement des attaques réseau à partir d’indicateurs de compromission tout en évitant la centralisation des données, en supprimant le point de défaillance unique d’un serveur central, en assurant la traçabilité im-

muable des contributions de chaque participant, et en protégeant les paramètres du modèle échangés contre les risques de rétro-ingénierie ?

Cette problématique se décompose en quatre sous-questions techniques traitées successivement dans ce rapport :

- Quelle architecture de réseau de neurones permet de détecter efficacement plusieurs classes d’attaques à partir de flux réseau tabulaires ?
- Comment organiser l’apprentissage collaboratif entre plusieurs clients sans dépendre d’un serveur central, tout en conservant une qualité de convergence acceptable ?
- Quelle technologie permet d’enregistrer de façon immuable et vérifiable les échanges entre les clients, sans réintroduire un point de centralisation ?
- Quels mécanismes cryptographiques permettent de protéger les poids du modèle échangés contre une reconstruction partielle des données d’entraînement d’origine ?

1.3 Objectifs du projet

Les objectifs techniques poursuivis par ce projet sont les suivants :

- Développer un modèle hybride CNN-LSTM bidirectionnel pour la détection multi-classes d’attaques réseau à partir du jeu de données CIC-IDS2018.
- Mettre en place un apprentissage fédéré entre quatre clients, selon deux stratégies de partitionnement des données : un partitionnement IID (équilibré) et un partitionnement non-IID (déséquilibré, simulé par une distribution de Dirichlet), afin d’évaluer la robustesse du système dans des conditions réalistes.
- Supprimer le serveur central d’agrégation : chaque client calcule lui-même la moyenne des poids reçus de l’ensemble des participants (agrégation FedAvg pair-à-pair), garantissant qu’aucun nœud unique ne constitue un point de défaillance.
- Déployer une blockchain privée Hyperledger Fabric afin de tracer, de manière immuable et vérifiable, chaque échange de poids entre les quatre clients, round par round.
- Étudier la protection des poids du modèle contre le reverse engineering à l’aide de deux mécanismes : le chiffrement homomorphe (TenSEAL) pour une agrégation chiffrée (preuve de concept), et la confidentialité différentielle (Opacus) pour l’entraînement local.
- Évaluer précisément l’impact de la confidentialité différentielle sur les performances du modèle, afin de quantifier le compromis entre protection de la vie privée et qualité de détection.

1.4 Contributions attendues

À l’issue de ce projet, les contributions suivantes sont attendues :

- Une architecture complète et fonctionnelle combinant intelligence artificielle, apprentissage fédéré décentralisé, blockchain et sécurité des modèles, démontrée de bout en bout.
- Une expérimentation reproductible sur un jeu de données public et reconnu par la communauté scientifique en cybersécurité (CIC-IDS2018).
- Une analyse comparative entre un scénario sans aucune sécurité et un scénario avec confidentialité différentielle, mettant en évidence le compromis entre sécurité et performance.
- Une démonstration concrète de la traçabilité immuable des échanges via une blockchain réellement déployée, et non simulée par une structure de données en mémoire.

Conclusion

Ce chapitre a permis de cadrer le projet, de poser la problématique centrale et de préciser les objectifs scientifiques et techniques poursuivis. Le chapitre suivant présente l’état de l’art et les fondements théoriques nécessaires à la compréhension de la solution proposée.

2. État de l’art et fondements théoriques

Introduction

Ce chapitre présente les concepts théoriques mobilisés dans ce projet : les indicateurs de compromission et la détection d’intrusions, les architectures de deep learning adaptées au trafic réseau, l’apprentissage fédéré, la technologie blockchain Hyperledger Fabric, et les mécanismes de protection contre la rétro-ingénierie des modèles.

2.1 Indicateurs de compromission et détection d’intrusions

2.1.1 Définition des IoC

Un indicateur de compromission (IoC) est un artefact observable – une adresse IP, un motif de paquet, une durée de flux anormale, un volume de connexions atypique – qui signale, avec un certain degré de confiance, la présence d’une activité malveillante sur un système ou un réseau. Contrairement à une signature d’attaque figée, un IoC peut être de nature statistique ou comportementale, ce qui le rend exploitable par des modèles d’apprentissage automatique entraînés à reconnaître des motifs dans le trafic plutôt que des règles écrites manuellement.

2.1.2 Rôle des IoC dans l’identification d’activités malveillantes

Dans ce projet, les IoC sont représentés par les caractéristiques (*features*) extraites des flux réseau du jeu de données CIC-IDS2018 : durée du flux, nombre de paquets échangés, taille moyenne des paquets, drapeaux TCP, intervalles entre paquets, etc. Ces caractéristiques, combinées, permettent au modèle de distinguer un trafic normal (*Benign*) d’un trafic associé à une attaque (par exemple force brute FTP ou SSH).

2.1.3 Détection par signatures, anomalies et apprentissage automatique

Trois grandes familles d’approches coexistent en détection d’intrusions :

- **Détection par signatures** – repose sur une base de motifs d’attaques connus. Rapide mais incapable de détecter une attaque inédite (*zero-day*).
- **Détection par anomalies** – modélise le comportement normal et signale tout écart significatif. Plus généraliste mais plus sujette aux faux positifs.

- **Détection par apprentissage automatique** – apprend directement, à partir de données étiquetées, à distinguer les classes de trafic. C'est l'approche retenue dans ce projet, avec un modèle de deep learning supervisé.

2.2 Deep learning pour la détection d'attaques réseau

2.2.1 Intérêt des architectures CNN

Les réseaux de neurones convolutifs (CNN), historiquement développés pour le traitement d'images, sont également efficaces sur des données tabulaires lorsqu'on les traite comme une séquence unidimensionnelle de caractéristiques. Une couche de convolution 1D (Conv1d) appliquée sur le vecteur de caractéristiques d'un flux réseau permet d'extraire des motifs locaux – des combinaisons de caractéristiques adjacentes qui, ensemble, sont discriminantes pour une classe d'attaque donnée.

2.2.2 Apport des LSTM et BiLSTM

Les réseaux récurrents de type *Long Short-Term Memory* (LSTM) sont conçus pour modéliser des dépendances séquentielles. Dans ce projet, la sortie de la couche convolutive est traitée comme une séquence le long de l'axe des caractéristiques réduites, et un LSTM bidirectionnel (BiLSTM) apprend les dépendances dans les deux sens de cette séquence, doublant la capacité de représentation par rapport à un LSTM unidirectionnel.

2.2.3 Architecture hybride retenue

Le modèle utilisé dans ce projet combine ces deux blocs : une couche convolutive (Conv1d + ReLU + MaxPool1d) qui réduit et structure les caractéristiques d'entrée, suivie d'un BiLSTM qui modélise les dépendances entre les motifs extraits, et enfin un classificateur dense (Linear + ReLU + Dropout + Linear) qui produit la prédiction finale parmi les classes d'attaques. Cette architecture est détaillée au chapitre 5.

2.3 Apprentissage fédéré

2.3.1 Principe général

L'apprentissage fédéré (*Federated Learning*, FL) est un paradigme d'entraînement distribué dans lequel plusieurs participants entraînent un modèle sur leurs propres données locales, sans jamais transmettre ces données à un tiers. Seuls les paramètres du modèle (poids, gradients) sont échangés et agrégés, ce qui répond directement aux exigences de confidentialité des données réseau sensibles.

2.3.2 Algorithme FedAvg

L'algorithme *Federated Averaging* (FedAvg) consiste, à chaque round d'entraînement, à :

1. faire entraîner chaque client localement pendant un nombre fixé d'époques ;
2. collecter les poids résultants de chaque client ;
3. calculer la moyenne (éventuellement pondérée par la taille des données locales) de ces poids ;

$$\theta_{\text{avg}} = \frac{1}{4} \sum_{j=1}^4 \theta_j$$

4. redistribuer le modèle moyenné à chaque client pour le round suivant.

Dans l'architecture classique du FL, l'étape d'agrégation est réalisée par un serveur central. Dans ce projet, cette étape est au contraire réalisée de manière décentralisée : chaque client reçoit les poids de tous les autres clients et calcule lui-même la moyenne, supprimant ainsi le rôle du serveur central (voir section 5.6).

2.3.3 Partitionnement IID et non-IID

Le partitionnement des données entre les clients influence fortement la convergence de l'apprentissage fédéré :

- **Partitionnement IID** (*Independent and Identically Distributed*) – chaque client reçoit une proportion identique de chaque classe. C'est le scénario le plus favorable à la convergence, utilisé comme référence.
- **Partitionnement non-IID** – chaque client reçoit une distribution de classes différente, simulée dans ce projet par une distribution de Dirichlet de paramètre $\alpha = 0,5$. Plus α est petit, plus la distribution entre clients est déséquilibrée. Ce scénario est plus représentatif des conditions réelles, où chaque organisation observe un trafic différent.

2.4 Blockchain et traçabilité décentralisée

2.4.1 Intérêt d'une blockchain privée pour la traçabilité

Dans un système fédéré décentralisé, la suppression du serveur central pose une nouvelle question : comment garantir que les échanges de poids entre les clients sont historisés de façon fiable, vérifiable par tous les participants, et impossible à modifier après coup ? Une blockchain privée répond précisément à ce besoin en offrant un registre (*ledger*) distribué, répliqué sur plusieurs nœuds, dans lequel chaque transaction est validée par un mécanisme de consensus puis ajoutée de façon immuable.

2.4.2 Hyperledger Fabric

Hyperledger Fabric est une plateforme de blockchain permissionnée (à accès restreint, contrairement aux blockchains publiques comme Bitcoin ou Ethereum), développée sous l'égide de la Linux Foundation. Elle est particulièrement adaptée aux cas d'usage d'entreprise et de recherche où les participants sont connus et identifiés, ce qui correspond exactement au contexte de ce projet : quatre clients identifiés, chacun disposant d'une identité cryptographique propre.

Les éléments clés de l'architecture Fabric mobilisés dans ce projet sont :

- **Les peers** – nœuds qui maintiennent une copie du registre et exécutent le chaincode pour valider les transactions.
- **L'orderer** – service qui ordonne les transactions validées avant leur ajout au registre, garantissant un ordre cohérent entre tous les peers.
- **Les organisations (MSP)** – chaque organisation dispose de son propre fournisseur de services d'adhésion (*Membership Service Provider*), qui gère les identités cryptographiques de ses membres via une autorité de certification (CA).
- **Le chaincode** – le programme (smart contract) qui définit la logique métier exécutée sur le réseau ; dans ce projet, il est écrit en JavaScript et expose les fonctions d'enregistrement et de consultation des échanges entre clients.
- **Le canal (*channel*)** – un sous-réseau logique sur lequel les transactions sont isolées ; toutes les transactions de ce projet sont enregistrées sur un canal unique nommé `mychannel`.

2.4.3 Rôle du chaincode dans l'enregistrement des échanges

Le chaincode développé pour ce projet (détaillé au chapitre 5) expose une fonction `RecordExchange`, invoquée par chaque client après chaque round d'agrégation, qui enregistre sur le registre le numéro de round, l'identité du client émetteur, l'identité du client destinataire, le hash SHA-256 des poids agrégés, le mode de partitionnement (IID ou non-IID) et un horodatage. Une fonction complémentaire `GetAllTransactions` permet d'interroger l'historique complet, démontrant la traçabilité du processus.

2.5 Protection contre le reverse engineering

2.5.1 Le risque de rétro-ingénierie des modèles fédérés

Même en l'absence de transmission directe des données, les poids d'un modèle entraîné localement peuvent, dans certaines conditions, encoder suffisamment d'information sur les données d'entraînement pour permettre une reconstruction partielle de celles-ci – un

risque documenté dans la littérature sur l'apprentissage fédéré sous le terme de *model inversion* ou *reverse engineering*. Ce projet mobilise deux mécanismes complémentaires pour limiter ce risque.

2.5.2 Chiffrement homomorphe avec TenSEAL

Le chiffrement homomorphe permet d'effectuer des opérations mathématiques (ici, une addition puis une division pour calculer une moyenne) directement sur des données chiffrées, sans jamais les déchiffrer. La bibliothèque TenSEAL implémente le schéma CKKS (*Cheon-Kim-Kim-Song*), adapté aux nombres à virgule flottante, ce qui correspond exactement à la nature des poids d'un réseau de neurones. Dans ce projet, TenSEAL est utilisé comme preuve de concept pour démontrer qu'une agrégation FedAvg peut être réalisée sur des poids chiffrés, bien que l'intégration complète dans le pipeline d'agrégation n'ait pas été menée à terme dans le temps imparti.

2.5.3 Confidentialité différentielle avec Opacus

La confidentialité différentielle (*Differential Privacy*, DP) consiste à injecter un bruit aléatoire calibré dans les gradients calculés pendant l'entraînement, de manière à rendre statistiquement difficile la détermination de la présence ou de l'absence d'un exemple particulier dans les données d'entraînement à partir des poids résultants. La bibliothèque Opacus, développée pour PyTorch, implémente ce mécanisme via un moteur de confidentialité (*PrivacyEngine*) qui ajoute du bruit gaussien aux gradients après les avoir écrêtés (*gradient clipping*). Le niveau de protection obtenu est quantifié par un budget de confidentialité noté ε (epsilon) : plus ε est petit, plus la protection est forte, mais plus l'apprentissage est perturbé.

2.5.4 Le compromis confidentialité-performance

L'un des résultats centraux de ce projet, détaillé au chapitre 6, est la mise en évidence empirique du compromis entre le niveau de protection différentielle (ε) et la capacité du modèle à converger. Ce compromis est un résultat attendu de la littérature sur la confidentialité différentielle appliquée au deep learning, mais sa quantification précise sur le jeu de données CIC-IDS2018 avec l'architecture CNN-LSTM retenue constitue une contribution expérimentale propre à ce projet.

Conclusion

Ce chapitre a posé les fondements théoriques nécessaires à la compréhension de l'architecture proposée : les IoC et la détection d'intrusions, les réseaux CNN-LSTM, l'apprentissage fédéré et son algorithme FedAvg, la blockchain Hyperledger Fabric, et les mécanismes de

protection par chiffrement homomorphe et confidentialité différentielle. Le chapitre suivant présente l'analyse fonctionnelle du système à concevoir.

3. Analyse fonctionnelle

Introduction

Ce chapitre présente l’analyse fonctionnelle du système à concevoir : les besoins fonctionnels et non fonctionnels, les cas d’usage identifiés, et une décomposition fonctionnelle du système selon la méthode FAST (*Function Analysis System Technique*).

3.1 Expression des besoins

3.1.1 Besoins fonctionnels

Le système doit permettre :

- le chargement et le nettoyage automatisé du jeu de données réseau CIC-IDS2018 ;
- le partitionnement des données entre quatre clients selon une stratégie IID et une stratégie non-IID configurables ;
- l’entraînement local d’un modèle CNN-LSTM par chaque client sur sa partition de données ;
- l’échange des poids entraînés entre les quatre clients et le calcul décentralisé de leur moyenne (FedAvg pair-à-pair) ;
- l’enregistrement immuable, sur une blockchain Hyperledger Fabric, de chaque échange de poids entre clients ;
- la consultation de l’historique complet des transactions enregistrées sur la blockchain ;
- l’application, à la demande, d’une couche de chiffrement homomorphe (TenSEAL) pour démontrer une agrégation chiffrée ;
- l’application, à la demande, d’une couche de confidentialité différentielle (Opacus) lors de l’entraînement local ;
- l’évaluation quantitative de la performance du modèle (F1-score, exactitude) à chaque round, pour chaque client.

3.1.2 Besoins non fonctionnels

- **Résilience** – absence de point de défaillance unique ; aucun composant central ne doit être indispensable au fonctionnement du système.

- **Traçabilité** – chaque échange de poids doit être attribuable à un client identifié et horodaté de façon vérifiable.
- **Confidentialité** – les données brutes de chaque client ne doivent jamais quitter son environnement local.
- **Reproductibilité** – les expérimentations doivent pouvoir être rejouées avec des graines aléatoires fixées.
- **Portabilité** – le système doit pouvoir s'exécuter sur une machine virtuelle Linux standard (Kali Linux), sans dépendance à un environnement cloud propriétaire.

3.2 Cas d'usage

3.2.1 Acteurs du système

- **Client fédéré** – entité disposant de ses propres données réseau, qui entraîne localement le modèle et soumet ses échanges à la blockchain.
- **Réseau Hyperledger Fabric** – infrastructure de traçabilité qui valide et enregistre les transactions soumises par les clients.
- **Analyste / encadrant** – utilisateur qui consulte les résultats d'entraînement et l'historique des transactions pour évaluer le système.

3.2.2 Cas d'usage principaux

- **UC1 – Entraîner localement le modèle** – un client charge sa partition de données et entraîne le modèle CNN-LSTM pendant un nombre fixé d'époques.
- **UC2 – Agréger les poids reçus** – un client reçoit les poids des trois autres clients et calcule la moyenne FedAvg, sans intervention d'un serveur central.
- **UC3 – Soumettre une transaction de traçabilité** – un client calcule le hash des poids agrégés et invoque le chaincode Fabric pour l'enregistrer sur le registre.
- **UC4 – Consulter l'historique des transactions** – l'analyste interroge le registre Fabric pour obtenir la liste complète des échanges, filtrée par round ou par client.
- **UC5 – Chiffrer les poids avec TenSEAL (démonstration)** – un client chiffre ses poids avec TenSEAL, sans pour autant les utiliser dans l'agrégation.
- **UC6 – Entraîner avec confidentialité différentielle** – un client active le moteur Opacus, qui injecte du bruit dans les gradients pendant l'entraînement local.
- **UC7 – Évaluer la performance du modèle** – à la fin de chaque round, le F1-score et l'exactitude de chaque client sont calculés et comparés à la baseline locale.

3.3 Décomposition fonctionnelle (méthode FAST)

La méthode FAST permet de décomposer la fonction globale du système en fonctions principales puis en fonctions techniques associées.

3.3.1 Fonction globale

Détecter des attaques réseau à partir d'indicateurs de compromission, de façon décentralisée, traçable et protégée contre la rétro-ingénierie des modèles partagés.

3.3.2 Fonctions principales et fonctions techniques

TABLE 3.1 – Fonctions principales et fonctions techniques associées

Fonction principale	Fonctions techniques associées
FP1 – Préparer les données	Nettoyer le jeu de données, encoder les variables catégorielles, normaliser les caractéristiques et partitionner entre les quatre clients (IID / non-IID).
FP2 – Entraîner le modèle localement	Construire l'architecture CNN-LSTM bidirectionnelle, optimiser les poids par descente de gradient (Adam) et évaluer le F1-score local.
FP3 – Fédérer l'apprentissage sans serveur central	Diffuser les poids locaux à l'ensemble des clients, calculer la moyenne FedAvg de façon décentralisée et répéter le processus sur plusieurs rounds.
FP4 – Tracer les échanges de façon immuable	Calculer le hash SHA-256 des poids agrégés, soumettre une transaction au chaincode Fabric, valider la transaction par les peers et enregistrer le résultat sur le registre.
FP5 – Protéger les poids échangés	Chiffrer les poids par schéma CKKS (TenSEAL) à titre de démonstration et injecter du bruit différentiel dans les gradients (Opacus) pendant l'entraînement.
FP6 – Évaluer et comparer	Mesurer le F1-score et l'exactitude par client et par round, puis comparer les scénarios IID / non-IID et sécurisé / non sécurisé.

Conclusion

Ce chapitre a formalisé les besoins fonctionnels et non fonctionnels du système, les principaux cas d'usage, et une décomposition fonctionnelle selon la méthode FAST. Cette analyse constitue la base de la charte de projet et de la conception architecturale présentées dans les chapitres suivants.

4. Charte du projet

Introduction

Ce chapitre présente la charte de gestion du projet : la matrice d'analyse des risques identifiés au cours du travail, le planning prévisionnel et réalisé, ainsi que la répartition des responsabilités au sein de l'équipe.

4.1 Matrice des risques

Le tableau 4.1 présente les principaux risques identifiés au cours du projet, leur probabilité d'occurrence, leur impact, et les mesures de mitigation mises en place. Certains risques se sont effectivement matérialisés au cours du projet, ce qui a directement motivé les évolutions techniques décrites au chapitre 5.

4.2 Matrice des risques

Le tableau 4.1 présente les principaux risques identifiés au cours du projet, leur probabilité d'occurrence, leur impact, ainsi que les mesures de mitigation mises en place. Certains risques se sont effectivement matérialisés au cours du projet, ce qui a directement motivé les évolutions techniques décrites au chapitre 5.

TABLE 4.1 – Matrice d'analyse des risques du projet

Risque identifié	Probabilité	Impact	Mesure de mitigation
Complexité de simulation d'une blockchain publique de type Ethereum/Besu dans un délai court	Élevée	Élevé	Migration vers Hyperledger Fabric, mieux outillé pour un déploiement local rapide via le <i>test-network</i> officiel.
Indisponibilité ou instabilité de l'environnement Google Colab pour des sessions longues	Moyenne	Moyen	Migration de l'environnement d'expérimentation vers une machine virtuelle Kali Linux locale, offrant un contrôle complet de l'environnement d'exécution.
Suite à la page suivante			

Table 4.1 – Suite de la matrice des risques

Risque identifié	Probabilité	Impact	Mesure de mitigation
Déséquilibre extrême des classes dans le partitionnement non-IID empêchant l'apprentissage de la classe minoritaire	Élevée	Élevé	Réduction du paramètre α de la distribution de Dirichlet et analyse détaillée de son impact sur les performances dans le chapitre des résultats.
Budget de confidentialité ε trop faible bloquant totalement la convergence du modèle sécurisé	Élevée	Élevé	Évaluation de plusieurs valeurs du paramètre <code>noise_multiplier</code> afin d'obtenir un compromis satisfaisant entre confidentialité et précision du modèle.
Incompatibilité entre la couche <code>nn.LSTM</code> de PyTorch et le mécanisme de confidentialité différentielle d'Opacus	Moyenne	Élevé	Remplacement de <code>nn.LSTM</code> par <code>opacus.layers.DPLSTM</code> , compatible avec le calcul des gradients par échantillon.
Retard dans la maîtrise des outils Hyperledger Fabric (Docker, chaincode, SDK) compte tenu du délai du projet	Moyenne	Élevé	Réduction du périmètre de la plateforme à deux organisations Hyperledger Fabric avec quatre identités clientes, au lieu de quatre organisations complètes.
Ressources matérielles insuffisantes pour exécuter simultanément Hyperledger Fabric et PyTorch	Faible	Moyen	Allocation de 16 Go de mémoire vive à la machine virtuelle et exécution de l'entraînement uniquement sur CPU.

4.3 Planning prévisionnel et réalisé

Le planning a été organisé en six phases. Le tableau 4.2 présente la planification sous forme de diagramme de Gantt simplifié, les semaines étant numérotées de S1 à S12.

TABLE 4.2 – Planning du projet (diagramme de Gantt simplifié)

Phase	S1-2	S3-4	S5-7	S8-9	S10-11	S12
Cadrage, choix du modèle, collecte des données	■					
Prétraitement et partitionnement IID / non-IID		■				
Entraînement local et apprentissage fédéré décentralisé			■			
Migration et déploiement Hyperledger Fabric				■		
Intégration TenSEAL et Opacus, analyse du compromis					■	
Rédaction du rapport et préparation de la soutenance						■

4.4 Répartition des responsabilités

- **Abiza Fadwa** – collecte et prétraitement des données, nettoyage du jeu de données CIC-IDS2018, partitionnement IID et non-IID, validation statistique des distributions entre clients.
- **El Aouni Ghizlane** – conception et implémentation du modèle CNN-LSTM, entraînement local par client, implémentation de l'apprentissage fédéré décentralisé et de l'agrégation FedAvg pair-à-pair.
- **El Kadiri Boutchich Radia** – déploiement du réseau Hyperledger Fabric, développement du chaincode JavaScript, intégration des couches de sécurité TenSEAL et Opacus.
- **Équipe complète** – définition de l'objectif du projet, choix et validation des outils, intégration finale des composants, évaluation des résultats, rédaction du rapport et préparation de la soutenance.

Conclusion

Ce chapitre a présenté la charte de gestion du projet, en mettant en évidence les risques anticipés – dont plusieurs se sont effectivement réalisés et ont motivé des évolutions techniques majeures – ainsi que le planning et la répartition des responsabilités au sein

de l'équipe. Le chapitre suivant approfondit les fondements théoriques nécessaires à la conception de l'architecture retenue.

5. Conception et implémentation

Introduction

Ce chapitre présente la conception détaillée de l’architecture retenue, ainsi que son implémentation technique : le modèle d’intelligence artificielle, l’apprentissage fédéré décentralisé, le déploiement de la blockchain Hyperledger Fabric, et l’intégration des couches de sécurité. Une attention particulière est portée aux évolutions par rapport au plan initial du projet, justifiées par des contraintes de faisabilité pratique.

5.1 Vue d’ensemble de l’architecture

L’architecture du système repose sur deux niveaux complémentaires, illustrés à la figure D.1.

- **Niveau 1 – apprentissage fédéré décentralisé** – quatre clients entraînent localement le modèle CNN-LSTM sur leur partition de données. À chaque round, ils échangent leurs poids en pair-à-pair et chaque client calcule lui-même la moyenne FedAvg à partir des poids reçus de tous les autres, sans aucun serveur central ni agrégation externe.
- **Niveau 2 – traçabilité immuable** – une fois l’agrégation locale effectuée, chaque client calcule un hash des poids agrégés et soumet cette empreinte comme transaction à un chaincode déployé sur un réseau Hyperledger Fabric. La transaction est validée et enregistrée de façon immuable sur le registre distribué, garantissant ainsi la traçabilité de chaque échange sans que la blockchain ait à connaître les poids eux-mêmes.

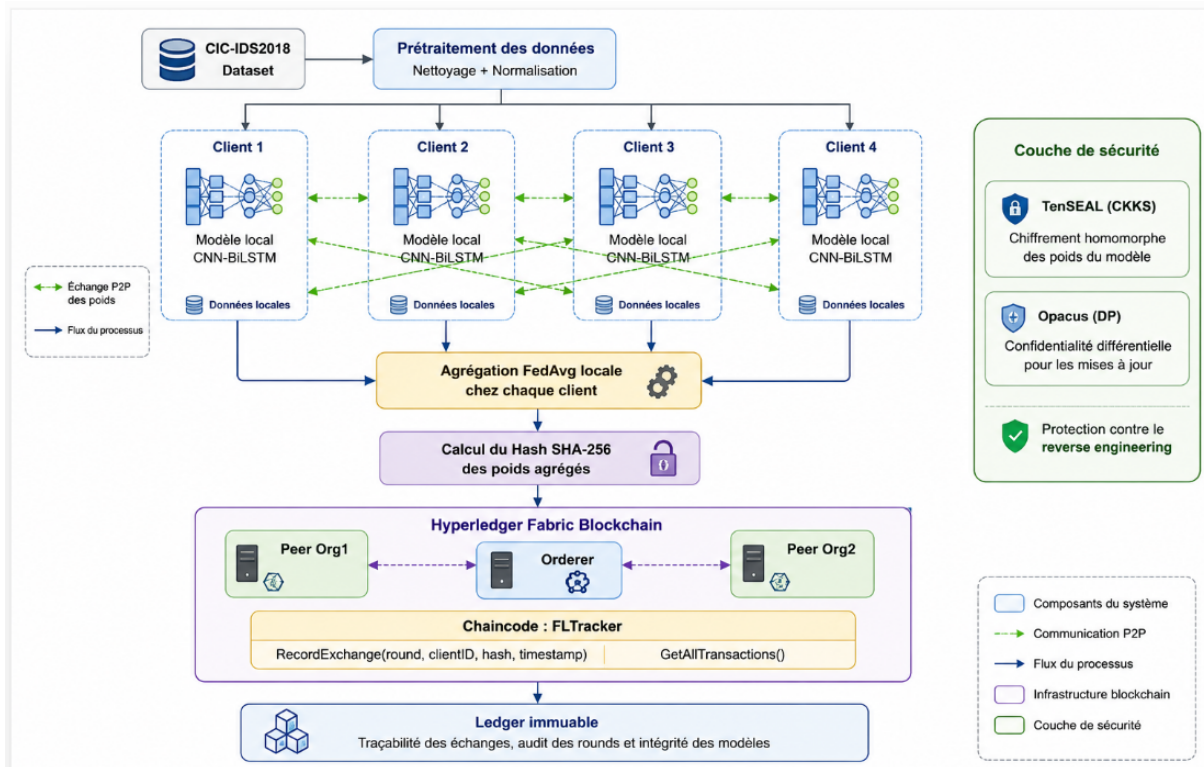


FIGURE 5.1 – Architecture globale du système – apprentissage fédéré décentralisé et traçabilité blockchain

5.2 Évolutions par rapport au plan initial

Le tableau 5.1 synthétise les principales évolutions techniques apportées au plan initial du projet, ainsi que leur justification.

5.3 Évolutions techniques

Le tableau 5.1 présente les principales évolutions techniques apportées au projet par rapport au plan initial, ainsi que les motivations ayant conduit à ces choix.

TABLE 5.1 – Évolutions techniques par rapport au plan initial

Élément du plan initial	Solution retenue	Justification
Hyperledger Besu avec consensus IBFT 2.0	Hyperledger Fabric utilisant le <i>test-network</i> officiel	Outillage de déploiement local plus mature pour respecter les contraintes de temps du projet. La gestion des identités via le MSP est directement adaptée à une architecture composée de quatre clients identifiés.
Suite à la page suivante		

Table 5.1 – Suite du tableau

Élément du plan initial	Solution retenue	Justification
Smart contract Solidity avec Remix IDE / Hardhat	Chaincode JavaScript avec Fabric Gateway SDK	Choix cohérent avec l'écosystème Node.js utilisé par Hyperledger Fabric. Cette solution réduit la courbe d'apprentissage tout en facilitant l'intégration avec les services blockchain.
Apprentissage fédéré centralisé basé sur Flower	Apprentissage fédéré décentralisé pair-à-pair	Suppression du point de défaillance unique conformément aux recommandations de l'encadrement. Chaque client réalise localement l'agrégation des paramètres reçus.
Preuves de légitimité fondées sur HMAC ou Zero-Knowledge Proof (ZKP)	Calcul du hash SHA-256 des poids agrégés	Le calcul d'un hash garantit l'intégrité et la traçabilité des échanges enregistrés sur la blockchain. Les mécanismes ZKP sont conservés comme perspective d'évolution future.
Agrégation robuste de type Krum (tolérance byzantine)	Fonctionnalité non implémentée dans la version actuelle	Cette fonctionnalité a été exclue du périmètre afin de respecter les délais de réalisation. Elle constitue une amélioration envisageable dans les travaux futurs.
Environnement d'expérimentation Google Colab	Machine virtuelle Kali Linux locale	Le déploiement de Docker ainsi que l'exécution du réseau Hyperledger Fabric nécessitent un environnement local offrant davantage de contrôle que Google Colab.

5.4 Architecture du modèle CNN-LSTM

5.4.1 Flux des données dans le modèle

Le modèle CNN-LSTM bidirectionnel traite chaque flux réseau (vecteur de 77 caractéristiques après nettoyage) selon le flux suivant :

1. **Entrée** – vecteur de caractéristiques de dimension $[\text{batch}, 77]$.
2. **Redimensionnement** – ajout d'une dimension de canal : $[\text{batch}, 1, 77]$.
3. **Convolution 1D** – 64 filtres, noyau de taille 3, padding 1 : $[\text{batch}, 64, 77]$, suivie d'une activation ReLU et d'un *max pooling* de taille 2, réduisant la longueur de séquence à 39 : $[\text{batch}, 64, 39]$.
4. **Transposition** – réorganisation pour le LSTM : $[\text{batch}, 39, 64]$, où 39 devient la longueur de séquence et 64 le nombre de caractéristiques par pas de temps.
5. **BiLSTM** – 128 unités cachées par direction, bidirectionnel : $[\text{batch}, 39, 256]$ ($256 = 128 \times 2$ directions).
6. **Extraction du dernier pas de temps** – $[\text{batch}, 256]$.

7. **Classificateur dense** – `Linear(256 → 128), ReLU, Dropout(0.3), Linear(128 → num_classes)`.

5.4.2 Compatibilité avec la confidentialité différentielle

Un point d’implémentation important concerne la compatibilité entre la couche récurrente et le moteur de confidentialité différentielle Opacus. La couche standard `torch.nn.LSTM` de PyTorch n’expose pas les gradients par échantillon nécessaires au calcul du bruit différentiel. Elle a donc été remplacée par `opacus.layers.DPLSTM`, une implémentation fonctionnellement équivalente mais compatible avec le mécanisme de calcul de gradients par échantillon requis par Opacus.

```

1 from opacus.layers import DPLSTM
2
3 self.lstm = DPLSTM(
4     64, 128,
5     batch_first=True,
6     bidirectional=True
7 )

```

Listing 5.1 – Définition de la couche récurrente compatible Opacus

5.4.3 Validation de l’architecture

Avant tout entraînement fédéré, l’architecture a été validée unitairement en vérifiant que la forme de sortie correspond bien au nombre de classes attendu, et en comptabilisant le nombre total de paramètres entraînaibles du modèle (232 195 paramètres dans la configuration retenue, pour 77 caractéristiques d’entrée et 2 classes de sortie).

5.5 Préparation et partitionnement des données

5.5.1 Jeu de données CIC-IDS2018

Le jeu de données utilisé est CIC-IDS2018, qui contient des flux réseau étiquetés couvrant plusieurs familles d’attaques. Après nettoyage (suppression des valeurs infinies et manquantes, encodage des colonnes catégorielles telles que `Protocol`, normalisation des caractéristiques numériques via `StandardScaler`), le jeu de données exploité comprend 77 caractéristiques par flux et deux classes (Benign et attaque). Le jeu de données initial est limité à 100 000 lignes pour des raisons de temps d’exécution.

5.5.2 Partitionnement entre les quatre clients

Deux stratégies de partitionnement ont été implémentées et comparées :

- **Partitionnement IID** – pour chaque classe, les indices sont mélangés puis répartis en quatre parts égales, garantissant que chaque client reçoit une proportion quasi identique de chaque classe.
- **Partitionnement non-IID** – pour chaque classe, une distribution de Dirichlet de paramètre $\alpha = 0,5$ détermine la proportion attribuée à chaque client, produisant des partitions très déséquilibrées et représentatives d’un scénario réel où chaque client observe un trafic différent.

5.6 Apprentissage fédéré décentralisé pair-à-pair

5.6.1 Suppression du serveur central

Conformément à l’exigence de résilience formulée par l’encadrement, le système ne fait reposer l’agrégation FedAvg sur aucun serveur central. À chaque round, la séquence suivante est exécutée :

1. chaque client entraîne son modèle localement pendant trois époques sur sa partition de données ;
2. chaque client transmet son état de modèle (*state_dict*) aux trois autres clients ;
3. chaque client calcule lui-même, en local, la moyenne arithmétique des quatre états de modèle reçus (FedAvg) ;
4. chaque client met à jour son propre modèle avec cette moyenne, qui devient le point de départ du round suivant.

Cette conception garantit qu’aucun nœud unique n’est indispensable : si un client devient indisponible, les trois autres peuvent en principe poursuivre l’agrégation entre eux, contrairement à une architecture FL classique où la panne du serveur d’agrégation arrête l’ensemble du processus.

5.6.2 Extrait d’implémentation

```

1 # Après l'entraînement local de chaque client, on collecte les poids
2 tous_les_poids = [modele.state_dict() for modele in modeles]
3
4 for i in range(4):
5     # Calcul de la moyenne des poids de tous les clients
6     poids_moyen = {}
7     for key in tous_les_poids[0].keys():
8         poids_moyen[key] = torch.stack(
9             [p[key] for p in tous_les_poids]
10            ).mean(dim=0)
11     modeles[i].load_state_dict(poids_moyen)

```

```
12
13     # Calcul du hash pour la blockchain
14     poids_flat = torch.cat(
15         [p.flatten() for p in poids_moyen.values()]
16     ).numpy()
17     hash_poids = hashlib.sha256(
18         poids_flat[:1000].tobytes()
19     ).hexdigest()
20
21     # Enregistrement sur Fabric
22     fabric_connector.enregistrer_echange(
23         round_num=round_num,
24         emetteur="Reseau",
25         destinataire=f"Client{i+1}",
26         hash_poids=hash_poids,
27         mode_type=mode_type,
28         timestamp=datetime.now().isoformat()
29     )
```

Listing 5.2 – Agrégation FedAvg décentralisée (extrait de `main_5rounds.py`)

5.7 Déploiement du réseau Hyperledger Fabric

5.7.1 Choix d'architecture réseau

Compte tenu du délai imparti, le réseau Fabric repose sur le *test-network* officiel, structuré en deux organisations (Org1 et Org2), chacune disposant d'un peer et d'une autorité de certification, ainsi que d'un service d'ordonnancement (*orderer*). Plutôt que de configurer quatre organisations Fabric complètes – ce qui aurait demandé de réécrire l'ensemble de la configuration de consensus et des fichiers de certification, au prix d'un temps de mise en œuvre incompatible avec le délai du projet – les quatre clients de l'apprentissage fédéré sont représentés par quatre identités cryptographiques distinctes, enrôlées auprès des autorités de certification des deux organisations existantes. Chaque transaction soumise au chaincode est ainsi signée par l'identité du client émetteur, garantissant une attribution et une traçabilité réelles, sans nécessiter une infrastructure à quatre organisations.

5.7.2 Composants du réseau déployé

- Un **orderer** (`orderer.example.com`), responsable de l'ordonnancement des transactions avant leur ajout au registre.
- Deux **peers** (`peer0.org1.example.com`, `peer0.org2.example.com`), qui valident et exécutent le chaincode, et maintiennent chacun une copie du registre.

- Deux **autorités de certification** (`ca_org1`, `ca_org2`), qui délivrent les certificats des identités clients.
- Un **canal** (`mychannel`), sur lequel l'ensemble des transactions de traçabilité fédérée sont enregistrées.

```
(venv)-(kali@kali)-[~/fl-project]
$ docker ps
CONTAINER ID   IMAGE                                     COMMAND                                     CREATED        STATUS        PORTS
db0fb04fc18d   dev-peer0.org1.example.com-fltracker_1.0-d897f5a16beb52e55220e5b90fafa8822f111ed1a40e8a61a31731c31eecd2f5be606c8c7112c4ae248e04a0d516b25ac4daac424cebc "docker-entrypoint.s..." 30 hours ago   Up 30 hours
mple.com-fltracker_1.0-d897f5a16beb52e55220e5b90fafa8822f111ed1a40e8a61a31731c318d524fd
69cbbbc8b106   dev-peer0.org2.example.com-fltracker_1.0-d897f5a16beb52e55220e5b90fafa8822f111ed1a40e8a61a31731c31aa72cbe75bfca1c7d0951937ca0ce4bfd7d53fd32d6ea0de0f1b "docker-entrypoint.s..." 30 hours ago   Up 30 hours
mple.com-fltracker_1.0-d897f5a16beb52e55220e5b90fafa8822f111ed1a40e8a61a31731c318d524fd
f55f2df3a2d6   hyperledger/fabric-orderer:latest        "orderer"                                30 hours ago   Up 30 hours   0.0
, [::]:7050→7050/tcp, 0.0.0.0:7053→7053/tcp, [::]:7053→7053/tcp, 0.0.0.0:9443→9443/tcp, [::]:9443→9443/tcp
d80f4a8c3a51   hyperledger/fabric-peer:latest          "peer node start"                        30 hours ago   Up 30 hours   0.0
, [::]:9051→9051/tcp, 7051/tcp, 0.0.0.0:9445→9445/tcp, [::]:9445→9445/tcp
.com
a480d635d89c   hyperledger/fabric-peer:latest          "peer node start"                        30 hours ago   Up 30 hours   0.0
, [::]:7051→7051/tcp, 0.0.0.0:9444→9444/tcp, [::]:9444→9444/tcp
.com
fcc2d77e0f4e   hyperledger/fabric-ca:latest            "sh -c 'fabric-ca-se..."              30 hours ago   Up 30 hours   0.0
, [::]:8054→8054/tcp, 7054/tcp, 0.0.0.0:18054→18054/tcp, [::]:18054→18054/tcp
4fa3df42ee8d   hyperledger/fabric-ca:latest            "sh -c 'fabric-ca-se..."              30 hours ago   Up 30 hours   0.0
, [::]:7054→7054/tcp, 0.0.0.0:17054→17054/tcp, [::]:17054→17054/tcp
1c0bcfc23a85   hyperledger/fabric-ca:latest            "sh -c 'fabric-ca-se..."              30 hours ago   Up 30 hours   0.0
, [::]:9054→9054/tcp, 7054/tcp, 0.0.0.0:19054→19054/tcp, [::]:19054→19054/tcp
```

FIGURE 5.2 – Containers Docker actifs du réseau Hyperledger Fabric (sortie de `docker ps`)

5.8 Chaincode de traçabilité

5.8.1 Fonctions exposées

Le chaincode, développé en JavaScript et déployé sous le nom `fltracker`, expose les fonctions suivantes :

- `RecordExchange(round, clientEmetteur, clientDestinataire, hashPoids, modeType, timestamp)` – enregistre une transaction d'échange de poids entre deux clients pour un round donné.
- `GetTransaction(txId)` – retourne le détail d'une transaction identifiée par son identifiant.
- `GetAllTransactions()` – retourne l'historique complet des transactions enregistrées sur le registre.
- `GetTransactionsByRound(round)` – filtre l'historique par numéro de round.
- `GetTransactionsByClient(clientId)` – filtre l'historique par identité de client, émettrice ou destinataire.

5.8.2 Extrait du chaincode

```

1  async RecordExchange(ctx, round, clientEmetteur,
2      clientDestinataire, hashPoids, modeType, timestamp) {
3
4      const txId = ctx.stub.getTxID();
5
6      const transaction = {
7          docType: 'flExchange',
8          txId: txId,
9          round: parseInt(round),
10         clientEmetteur: clientEmetteur,
11         clientDestinataire: clientDestinataire,
12         hashPoids: hashPoids,
13         modeType: modeType,
14         timestamp: timestamp,
15         submittingMSP: ctx.clientIdentity.getMSPID(),
16         submittingID: ctx.clientIdentity.getID()
17     };
18
19     await ctx.stub.putState(
20         txId, Buffer.from(JSON.stringify(transaction))
21     );
22
23     return JSON.stringify(transaction);
24 }

```

Listing 5.3 – Fonction RecordExchange du chaincode fltracker

Chaque transaction enregistrée inclut non seulement le hash des poids et le round, mais aussi le *MSP* (l'organisation Fabric) et l'identité cryptographique exacte du client soumetteur, obtenus directement depuis le contexte d'exécution du chaincode (`ctx.clientIdentity`). Cette information est fournie par l'infrastructure Fabric elle-même et ne peut être falsifiée par le client, ce qui constitue la garantie d'attribution recherchée.

5.8.3 Calcul du hash des poids agrégés

Avant chaque soumission de transaction, le hash SHA-256 des poids agrégés est calculé côté client, en Python, à partir des paramètres aplatis du modèle :

```

1  poids_flat = torch.cat(
2      [p.flatten() for p in poids_moyen.values()]
3  ).numpy()
4
5  poids_hash = hashlib.sha256(
6      poids_flat[:1000].tobytes()
7  ).hexdigest()

```

Listing 5.4 – Calcul du hash des poids avant soumission à Fabric

5.8.4 Pont entre le pipeline Python et le réseau Fabric

Le SDK officiel Fabric Gateway étant principalement maintenu pour Node.js et Go, le pipeline d'apprentissage fédéré – écrit en Python – communique avec le réseau Fabric par l'intermédiaire d'une classe `FabricConnector` qui encapsule les appels à la commande `peer` en ligne de commande. Le script Python invoque cette classe à chaque round, pour chaque client, en utilisant les sous-processus pour exécuter les commandes `peer chaincode invoke` et `peer chaincode query`.

```

1 class FabricConnector:
2     def __init__(self, test_network_path):
3         self.path = test_network_path
4         # Configuration des variables d'environnement pour l'invocation
           peer
5         self.env = os.environ.copy()
6         self.env['FABRIC_CFG_PATH'] = f"{self.path}/../config/"
7         # ...
8     def enregistrer_echange(self, round_num, emetteur, destinataire,
9                             hash_poids, mode_type, timestamp):
10
11         cmd = [
12             'peer', 'chaincode', 'invoke',
13             '-C', 'mychannel',
14             '-n', 'fltracker',
15             '-c', json.dumps({
16                 'function': 'RecordExchange',
17                 'Args': [str(round_num), str(emetteur), str(destinataire),
18                         str(hash_poids), str(mode_type), str(timestamp)]
19             })
20         ]
21         result = subprocess.run(cmd, env=self.env, capture_output=True,
22                                 text=True, cwd=self.path)
23         # ...

```

Listing 5.5 – Extrait de la classe `FabricConnector`

5.9 Intégration des couches de sécurité

5.9.1 Chiffrement homomorphe avec TenSEAL

Un contexte de chiffrement CKKS est créé avec un degré de modulus polynomial de 8192 et des tailles de coefficients `[60, 40, 40, 60]`, offrant un compromis adapté entre

précision numérique et performance de calcul. Dans le script `main_tenseal_only.py`, les poids de chaque client sont chiffrés individuellement, puis additionnés homomorphiquement – c’est-à-dire sans déchiffrement intermédiaire – avant d’être divisés par le nombre de clients pour obtenir la moyenne FedAvg chiffrée. Seul le résultat final est déchiffré, démontrant que le serveur (ou tout intermédiaire) n’a, à aucun moment, accès aux poids individuels en clair. Cette démonstration est effectuée sur un sous-ensemble des poids (les 50 premiers) pour des raisons de performance. L’intégration complète dans le pipeline d’agrégation n’a pas été réalisée dans le temps imparti, mais la preuve de concept valide la faisabilité.

5.9.2 Confidentialité différentielle avec Opacus

Le moteur de confidentialité `PrivacyEngine` d’Opacus est attaché au modèle, à l’optimiseur et au chargeur de données du client concerné, avec un `noise_multiplier` et une norme maximale de gradient (`max_grad_norm`) configurables. Dans le script `main_opacus_balanced.py`, la confidentialité différentielle est activée à partir du premier round avec un `noise_multiplier` de 2,5 et une norme de gradient de 1,0. À la fin de l’entraînement, le budget de confidentialité ϵ est calculé pour un δ fixé à 10^{-5} , et constitue la mesure quantitative du niveau de protection atteint.

```

1 moteur_privacy = PrivacyEngine()
2
3 modele_dp, optimiseur_dp, loader_dp = moteur_privacy.make_private(
4     module=modele_dp,
5     optimizer=optimiseur_dp,
6     data_loader=loader_dp,
7     noise_multiplier=2.5,
8     max_grad_norm=1.0,
9 )

```

Listing 5.6 – Activation d’Opacus sur le modèle CNN-LSTM

Conclusion

Ce chapitre a détaillé la conception et l’implémentation technique du système : l’architecture CNN-LSTM, la suppression du serveur central d’agrégation, le déploiement du réseau Hyperledger Fabric et de son chaincode, ainsi que l’intégration des couches de sécurité TenSEAL (preuve de concept) et Opacus. Le chapitre suivant présente et interprète les résultats expérimentaux obtenus à partir de cette implémentation.

6. Résultats et interprétation

Introduction

Ce chapitre présente les résultats expérimentaux obtenus à travers trois scénarios successifs, du moins sécurisé au plus protégé, exécutés sur la VM Kali Linux avec le réseau Hyperledger Fabric déployé. Ces résultats permettent de quantifier précisément le compromis entre la protection des poids échangés et la performance du modèle de détection.

6.1 Métriques d'évaluation

Les métriques retenues pour évaluer chaque client à chaque round sont :

- le **F1-score macro**, qui moyenne la performance sur l'ensemble des classes sans privilégier la classe majoritaire, ce qui est essentiel sur un jeu de données déséquilibré comme CIC-IDS2018 ;
- l'**exactitude** (*accuracy*), qui mesure la proportion globale de prédictions correctes ;
- le **budget de confidentialité** ε , calculé pour $\delta = 10^{-5}$, qui quantifie le niveau de protection différentielle atteint lorsque Opacus est activé.

6.2 Scénarios expérimentaux

Trois scripts ont été exécutés successivement, chacun correspondant à un niveau de sécurité différent, sur un sous-ensemble de 100 000 lignes du jeu de données CIC-IDS2018, réparties entre les quatre clients selon les stratégies IID et non-IID décrites au chapitre précédent.

TABLE 6.1 – Synthèse des scénarios expérimentaux et résultats obtenus

Script	Sécurité activée	F1 IID	F1 Non-IID	Observation
main_5rounds.py	Aucune	1.0000	1.0000	Convergence rapide en 3 rounds.
main_tenseal_only.py	TenSEAL (démonstration de chiffrement)	1.0000	1.0000	Aucune dégradation de performance liée au chiffrement ; preuve de concept validée.
main_opacus_balanced.py	Opacus + TenSEAL ($\epsilon \approx 0,49$)	0.4996	0.4996	Apprentissage bloqué dès le premier round ; bruit différentiel trop important.

6.3 Scénario 1 – Sans aucune sécurité (main_5rounds.py)

En l’absence de toute couche de protection, le modèle CNN-LSTM converge très rapidement vers un F1-score macro parfait (1.0), à la fois en partitionnement IID et non-IID, en seulement 3 à 4 rounds de fédération. Ce résultat confirme que l’architecture du modèle et la stratégie d’agrégation décentralisée FedAvg sont fonctionnelles et permettent une convergence efficace lorsque la confidentialité n’est pas une contrainte.

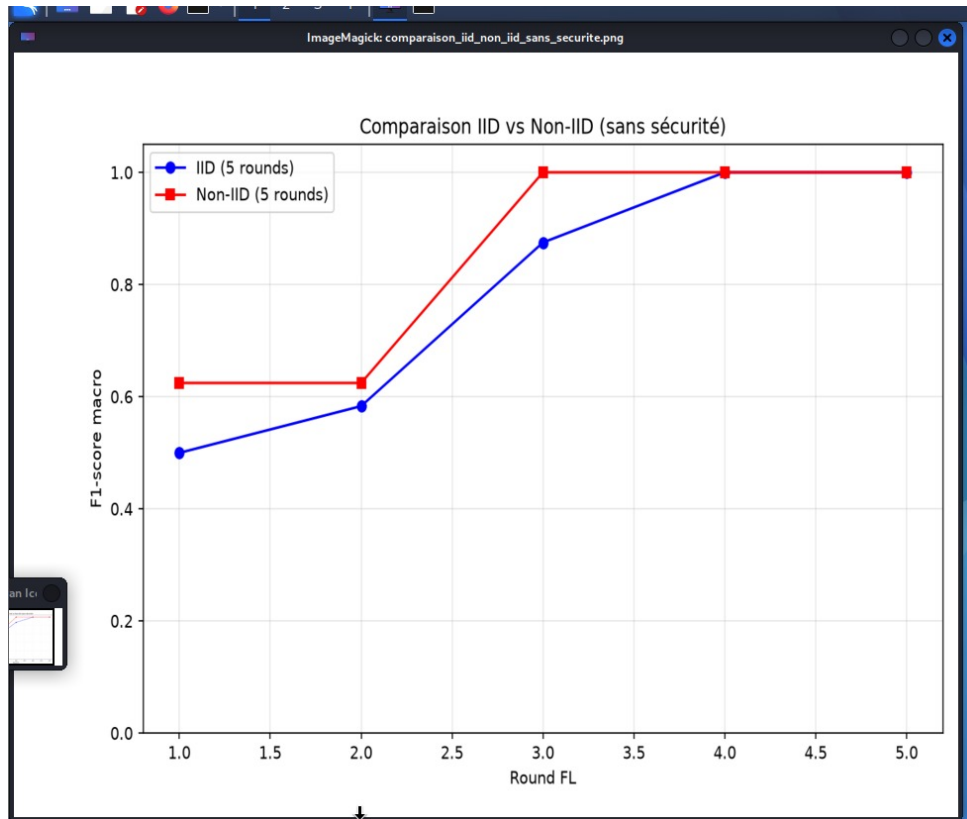


FIGURE 6.1 – Convergence du F1-score en partitionnement IID et NON IID sans sécurité (script `main_5rounds.py`)

6.4 Scénario 2 – Chiffrement homomorphe TenSEAL (démonstration) (`main_tenseal_only.py`)

L'ajout du chiffrement homomorphe CKKS (TenSEAL) dans le script dédié ne dégrade aucunement les performances obtenues : le F1-score reste à 1.0 dans les deux partitionnements. Ce résultat est attendu, car le chiffrement homomorphe est conçu pour préserver exactement le résultat du calcul effectué (ici, une moyenne), à une erreur d'arrondi numérique près, mesurée comme quasi nulle lors des tests. TenSEAL protège donc la confidentialité des poids individuels pendant l'agrégation sans coût en performance, ce qui en fait une couche de sécurité particulièrement intéressante à déployer systématiquement, même si son intégration complète dans le pipeline n'a pas été finalisée.

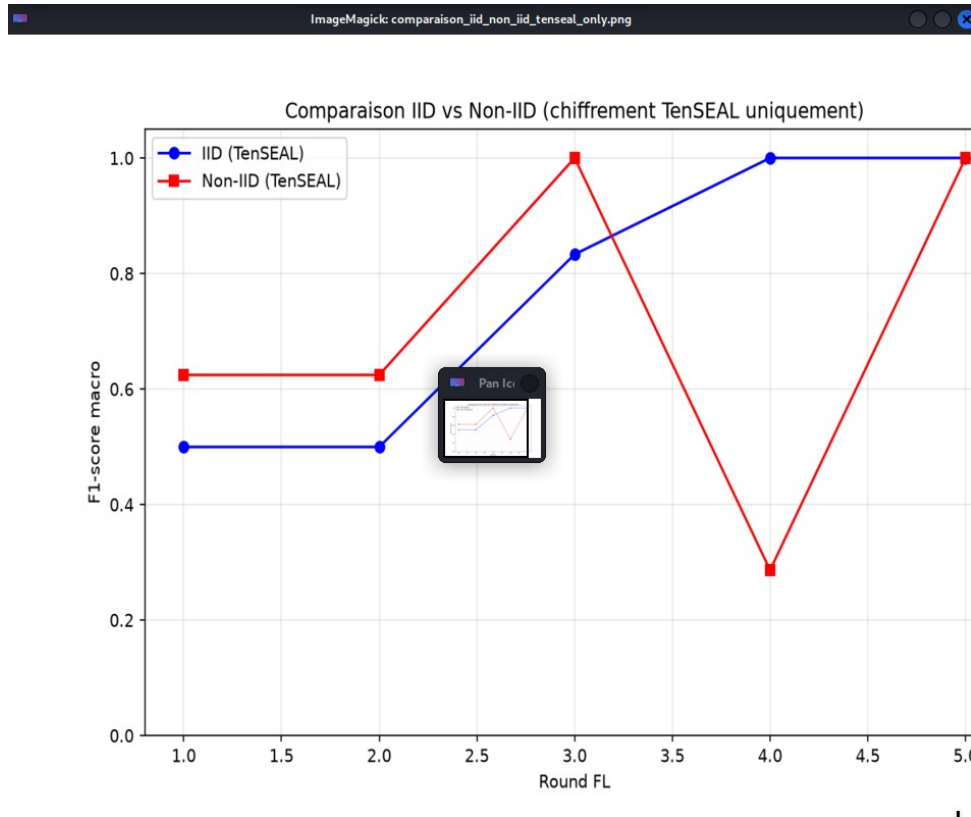


FIGURE 6.2 – Convergence du F1-score en partitionnement IID et NON IID avec sécurité (TenSEAL)(script `main_tenseal_only`)

6.5 Scénario 3 – Opacus avec budget de confidentialité strict ($\epsilon \approx 0,49$) (`main_opacus_balanced.py`)

Lorsque la confidentialité différentielle est activée avec un budget de confidentialité strict ($\epsilon \approx 0,49$) sur un sous-ensemble de 4000 échantillons par client, le F1-score macro reste bloqué à 0.4996 en IID et en non-IID, quel que soit le nombre de rounds exécutés. Ce blocage s'explique par le niveau de bruit injecté dans les gradients : pour atteindre un ϵ aussi faible, le `noise_multiplier` est élevé (2,5), ce qui noie le signal utile du gradient et empêche le modèle d'apprendre des motifs discriminants, en particulier pour les classes minoritaires d'attaques qui disposent déjà de peu d'exemples.

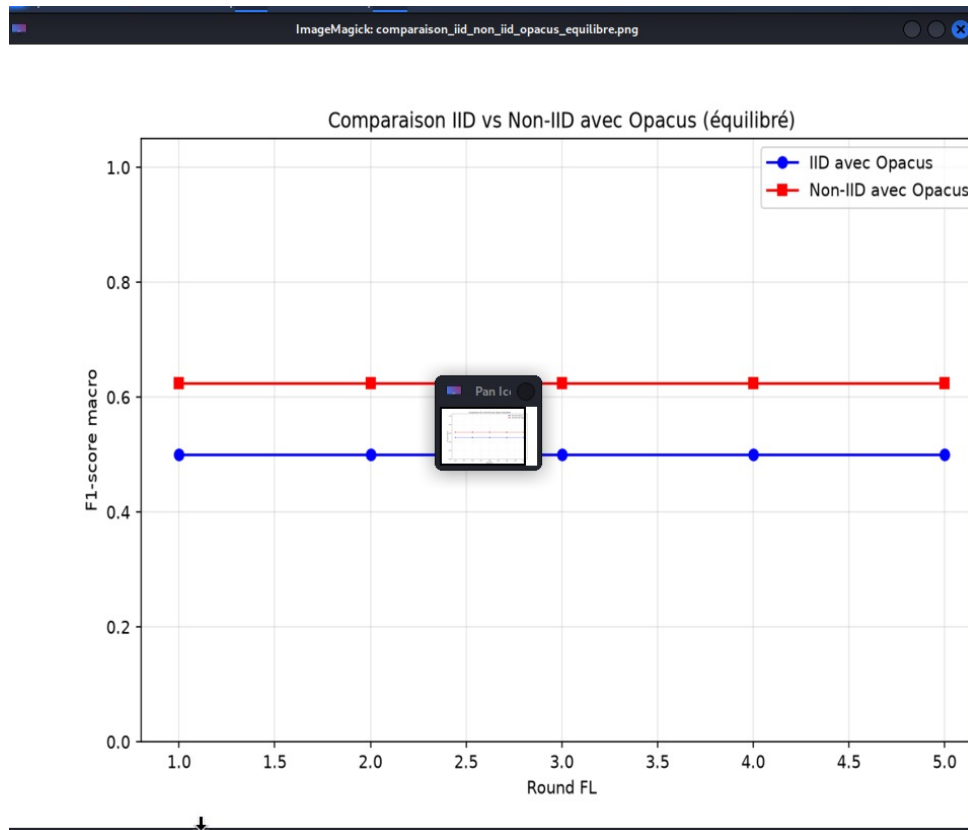


FIGURE 6.3 — Comparaison des F1-scores IID vs non-IID avec Opacus activé (script `main_opacus_balanced.py`)

6.6 Analyse du compromis confidentialité-performance

Les trois scénarios mettent en évidence un compromis net entre le niveau de protection différentielle et la capacité du modèle à converger :

- le chiffrement homomorphe (TenSEAL) constitue une protection « *gratuite* » en termes de performance, puisqu’il préserve exactement le résultat de l’agrégation ;
- la confidentialité différentielle (Opacus) impose en revanche un coût direct sur la performance, d’autant plus important que le budget ε est faible et que le nombre d’échantillons disponibles par client est restreint ;
- un budget ε compris entre 1,0 et 1,5, combiné à un nombre d’échantillons par client plus important que dans les scénarios testés, constituerait probablement un compromis plus équilibré entre protection de la vie privée et qualité de détection, ce qui est identifié comme piste d’amélioration au chapitre de conclusion.

6.7 Validation de la traçabilité blockchain

Indépendamment du niveau de sécurité testé, chaque round d’agrégation a donné lieu à l’enregistrement effectif d’une transaction par client sur le registre Hyperled-

ger Fabric, comme détaillé au chapitre 5. La consultation du registre via la fonction `GetAllTransactions` du chaincode confirme la présence de l'ensemble des transactions attendues, chacune associée à l'identité cryptographique du client émetteur, au round concerné, et au hash des poids agrégés.

```
(kali@kali)-[~/fabric-project/fabric-samples/test-network]
$ peer chaincode query -C mychannel -n fltracker -c '{"function":"GetAllTransactions","Args":[]}'

[{"docType":"flExchange","txId":"0369c0ec2363d82f2d077a8391712f76e78f2180d953d32b8bd930406a2c537e",
"round":1,"clientEmetteur":"Reseau","clientDestinataire":"Client4","hashPoids":"60e24667320183f45
6516c336b263909a65b1edea4f7604ee5d974f24ea1c34f","modeType":"IID","timestamp":"2026-06-20T08:37:24
.929564","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger/OU=
admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},{
"docType":"flExchange","txId":"03ae6b213c9a7e6f094ca582be610e4ee36a2ab41b29279f5355942505ab1e70",
"round":5,"clientEmetteur":"Reseau","clientDestinataire":"Client3","hashPoids":"29c8acf70dcd605cf01
13c0cfce4878e675db01e74b9e0d4c0e0a2074cc446e4","modeType":"NonIID","timestamp":"2026-06-20T08:19:0
2.936039","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger/OU=
admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},{
"docType":"flExchange","txId":"04fac5cc88752bec98d6ce2fd3554d31f890aada7538a3cb41b272dbd2c5e4ae",
"round":3,"clientEmetteur":"Reseau","clientDestinataire":"Client3","hashPoids":"ff58da7e03fd9e8f6f
26e72ff5f55444f71eeb2f2af1a9f4b9601551ce47d8d4","modeType":"NonIID","timestamp":"2026-06-20T08:13:
43.914664","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger/O
U=admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},{
"docType":"flExchange","txId":"09c599a69e807317e597577e9e6eb96179a2b31404f357940c6696f236e8fb71",
"round":1,"clientEmetteur":"Reseau","clientDestinataire":"Client2","hashPoids":"98fc8335b42467bea
8a71adc3138992d7c7decdf71ab5a2655c57e1dad2207f","modeType":"NonIID","timestamp":"2026-06-20T11:57
:34.044842","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger/
OU=admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},{
"docType":"flExchange","txId":"0b552c143d28f490c23c980f1eb4124d19a47cfc0c313c40b94951f742f4ef5b",
"round":5,"clientEmetteur":"Reseau","clientDestinataire":"Client2","hashPoids":"691df3bf228a2034
af00f22c7febe62552a9ab7bbfb165001cb05470628a3ca7","modeType":"NonIID","timestamp":"2026-06-20T09:2
1:54.790938","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger
/OU=admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},{
"docType":"flExchange","txId":"0c6ef66efc62bf48981c5c194b253c13cdc1a0b88623cbcee05492e0220aaee
f","round":3,"clientEmetteur":"Reseau","clientDestinataire":"Client3","hashPoids":"adba0aa12882a12
573eb9df7cc0f48ff98213fa0109b72d8e9e91bb58f5dedb0","modeType":"NonIID","timestamp":"2026-06-20T09:
16:08.863516","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledge
r/OU=admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.co
m"},{
"docType":"flExchange","txId":"0cf3dddeaf3154dac28b1a943e03495cd0b2eb4db3901a428f1f121da24500
a2","round":4,"clientEmetteur":"Reseau","clientDestinataire":"Client3","hashPoids":"19ed3f6c9390ae
9ba7f5b618eccbc55c87f7a9cd597ff3887ecef9f8c5876ed","modeType":"IID","timestamp":"2026-06-20T09:04
:30.661979","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger/
OU=admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},{
"docType":"flExchange","txId":"127ec7506ae2cc91cddccbd61bbfca7e325c2b687b48f76748a01a6714226a74",
"round":1,"clientEmetteur":"Reseau","clientDestinataire":"Client2","hashPoids":"992442f445a95309
294874f9fac50b5eff8663d0c48824f5f1aa0d0de1a2af9b","modeType":"IID","timestamp":"2026-06-20T08:37:2
4.709613","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger/OU=
admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},{
"docType":"flExchange","txId":"1398bda01b9ee032a7c49685dd1d22f0c43143afe5cb4356ec041c9830b42801",
```

FIGURE 6.4 – Aperçu de l'historique des transactions enregistrées sur le chaincode `fltracker` (requête `GetAllTransactions`)

```

(venv)-(kali@kali)-[~/fl-project]
$ python main_opacus_balanced.py

== Chargement du dataset ==
Shape brut : (100000, 80)
Features: 77, Classes: 2, Samples: 99997

== Partitionnement IID ==
Client 1: 25000 samples, classes: [ 15 24985]
Client 2: 24999 samples, classes: [ 29 24970]
Client 3: 24999 samples, classes: [ 33 24966]
Client 4: 24999 samples, classes: [ 38 24961]

== Partitionnement Non-IID (Dirichlet  $\alpha=0.5$ ) ==
Client 1: 6850 samples, classes: [ 31 6819]
Client 2: 26342 samples, classes: [ 53 26289]
Client 3: 771 samples, classes: [ 4 767]
Client 4: 66034 samples, classes: [ 27 66007]

===== EXÉCUTION IID AVEC OPACUS (ÉQUILIBRÉ) =====

— Round 1 (IID) —
👉 Activation de Differential Privacy (Opacus)
🔒 DP  $\epsilon$  = 0.4902
🔒 DP  $\epsilon$  = 0.4902
🔒 DP  $\epsilon$  = 0.4902
🔒 DP  $\epsilon$  = 0.4902
✅ Transaction enregistrée pour Client 1
✅ Transaction enregistrée pour Client 2
✅ Transaction enregistrée pour Client 3
✅ Transaction enregistrée pour Client 4
Client 1 - F1: 0.4997 | Acc: 0.9990
Client 2 - F1: 0.4997 | Acc: 0.9990
Client 3 - F1: 0.4995 | Acc: 0.9980
Client 4 - F1: 0.4995 | Acc: 0.9980
➔ F1 moyen du round: 0.4996

— Round 2 (IID) —
👉 Activation de Differential Privacy (Opacus)
🔒 DP  $\epsilon$  = 0.4902
🔒 DP  $\epsilon$  = 0.4902
🔒 DP  $\epsilon$  = 0.4902
🔒 DP  $\epsilon$  = 0.4902
✅ Transaction enregistrée pour Client 1
✅ Transaction enregistrée pour Client 2
✅ Transaction enregistrée pour Client 3
✅ Transaction enregistrée pour Client 4
Client 1 - F1: 0.4997 | Acc: 0.9990
Client 2 - F1: 0.4997 | Acc: 0.9990
Client 3 - F1: 0.4995 | Acc: 0.9980
Client 4 - F1: 0.4995 | Acc: 0.9980
➔ F1 moyen du round: 0.4996

```

FIGURE 6.5 — Extrait des journaux d'exécution du script `main_opacus_balanced.py` montrant le F1-score, le budget de confidentialité ϵ et les transactions blockchain

Conclusion

Ce chapitre a présenté les résultats expérimentaux obtenus sur trois scénarios de sécurité croissante. Il ressort de cette analyse que le chiffrement homomorphe TenSEAL ne dégrade pas la performance du modèle, tandis que la confidentialité différentielle Opacus impose un compromis significatif, d'autant plus marqué que le budget de confidentialité est strict et que les données disponibles par client sont limitées. La traçabilité des échanges sur la blockchain Hyperledger Fabric a, quant à elle, été validée dans l'ensemble des scénarios testés. Le chapitre suivant présente l'organisation du travail au sein de l'équipe et les livrables produits.

7. Organisation du travail, planning et livrables

Introduction

Ce chapitre récapitule l'organisation du travail au sein de l'équipe sur l'ensemble du projet, la pile technique complète effectivement utilisée, ainsi que les livrables produits à l'issue du travail.

7.1 Stack technique complète

Le tableau 7.1 récapitule l'ensemble des outils et technologies effectivement mobilisés dans la version finale du projet.

TABLE 7.1 – Pile technique complète du projet

Catégorie	Outils retenus
Jeu de données	CIC-IDS2018 (sous-ensemble de 100 000 lignes), détection binaire (Benign / Attaque)
Modèle d'IA	CNN-LSTM bidirectionnel, implémenté avec PyTorch
Apprentissage fédéré	Décentralisé pair-à-pair, agrégation FedAvg calculée localement par chaque client (sans Flower, sans serveur central)
Blockchain	Hyperledger Fabric, test-network (2 organisations, 4 identités clients)
Smart contract	Chaincode JavaScript (<code>fltracker</code>), déployé via le SDK Fabric
Sécurité	Opacus (confidentialité différentielle), TenSEAL (chiffrement homomorphe CKKS, preuve de concept)
Traçabilité	Hash SHA-256 des poids agrégés, enregistré sur le registre Fabric
Pont Python-Fabric	Classe <code>FabricConnector</code> utilisant les commandes <code>peer</code> en sous-processus
Environnement de travail	VM Kali Linux (4 Go de RAM), Docker, Node.js 18, Go 1.21, Python 3.10+

7.2 Répartition effective du travail

- **Abiza Fadwa** – collecte et prétraitement du jeu de données CIC-IDS2018, nettoyage, encodage et normalisation des caractéristiques, implémentation et validation statistique des partitionnements IID et non-IID entre les quatre clients.
- **El Aouni Ghizlane** – conception de l’architecture CNN-LSTM bidirectionnelle, implémentation de l’entraînement local par client, mise en place de l’apprentissage fédéré décentralisé et de l’agrégation FedAvg pair-à-pair, exécution des rounds expérimentaux IID et non-IID.
- **El Kadiri Boutchich Radia** – déploiement et configuration du réseau Hyperledger Fabric sur la VM Kali Linux, développement du chaincode JavaScript de traçabilité, mise en place du pont Python-Fabric (classe `FabricConnector`), intégration des couches de sécurité TenSEAL et Opacus.
- **Équipe complète** – cadrage initial de l’objectif du projet, arbitrage des évolutions techniques par rapport au plan initial, intégration finale de l’ensemble des composants, analyse conjointe des résultats expérimentaux, rédaction du rapport et préparation de la soutenance.

7.3 Livrables produits

- Le code source complet du pipeline d’intelligence artificielle et d’apprentissage fédéré décentralisé (modèle CNN-LSTM, partitionnement, entraînement, agrégation) : `main_5rounds.py`, `main_tenseal_only.py`, `main_opacus_balanced.py`.
- Le chaincode JavaScript de traçabilité (`fltracker.js`) et les scripts de déploiement sur le réseau Hyperledger Fabric.
- La classe `FabricConnector` pour l’interaction Python-Fabric.
- Les courbes de convergence et tableaux de résultats pour chaque scénario (sans sécurité, TenSEAL seul, Opacus + TenSEAL).
- Le présent rapport de projet de fin d’année.
- Un support de présentation et une démonstration en conditions réelles pour la soutenance.
- Dépôt GitHub : [PFA_FL_Blockchain_IoC](#)

Conclusion

Ce chapitre a récapitulé l’organisation du travail au sein de l’équipe, la pile technique complète effectivement mise en œuvre, et les livrables produits à l’issue de ce projet de fin

d'année. La conclusion générale qui suit revient sur les apports du projet et propose des perspectives d'amélioration.

Conclusion générale

Ce projet de fin d'année a permis de concevoir et d'implémenter un système complet de détection d'attaques réseau par indicateurs de compromission, combinant un modèle de deep learning hybride CNN-LSTM bidirectionnel, un apprentissage fédéré entièrement décentralisé entre quatre clients, une blockchain privée Hyperledger Fabric pour la traçabilité immuable des échanges, et deux mécanismes complémentaires de protection contre la rétro-ingénierie des modèles partagés : le chiffrement homomorphe TenSEAL (à titre de preuve de concept) et la confidentialité différentielle Opacus.

L'exigence centrale formulée par l'encadrement – l'absence de tout serveur central d'agrégation, afin de garantir la résilience du système face à une panne ou à une attaque ciblée – a été pleinement respectée : chaque client calcule lui-même la moyenne FedAvg des contributions de l'ensemble des participants. Cette suppression du point de coordination central a été compensée par la mise en place d'un véritable réseau Hyperledger Fabric, qui assure une traçabilité vérifiable et immuable de chaque échange entre les clients, répondant ainsi à l'exigence de confiance qu'impose la décentralisation.

Au cours du projet, plusieurs choix techniques initialement prévus ont dû évoluer pour des raisons de faisabilité pratique, dans le respect du délai imparti : la blockchain Hyperledger Besu a été remplacée par Hyperledger Fabric, le smart contract Solidity par un chaincode JavaScript, le framework Flower par une agrégation pair-à-pair entièrement décentralisée, et l'environnement Google Colab par une machine virtuelle Kali Linux locale, seule capable d'héberger l'infrastructure Docker nécessaire au réseau Fabric. Ces évolutions, documentées et justifiées tout au long de ce rapport, n'ont en rien altéré les objectifs scientifiques du projet : elles en ont, au contraire, permis une réalisation concrète et fonctionnelle dans le temps disponible.

Les résultats expérimentaux obtenus mettent en évidence un résultat scientifique clair : le chiffrement homomorphe protège la confidentialité des poids échangés sans coût de performance mesurable (preuve de concept), tandis que la confidentialité différentielle impose un compromis réel et quantifiable entre protection de la vie privée et qualité de détection, d'autant plus sévère que le budget de confidentialité ϵ est restreint et que les données disponibles par client sont limitées. Ce résultat illustre une tension bien documentée dans la littérature scientifique sur l'apprentissage fédéré sécurisé, et la contribution de ce projet est d'en avoir produit une mesure empirique précise sur un cas d'usage réel de

détection d'intrusions réseau.

Plusieurs perspectives d'amélioration se dégagent naturellement de ce travail. Sur le plan de la confidentialité différentielle, une exploration plus fine de la plage de budgets ε compris entre 1,0 et 1,5, combinée à une augmentation du nombre d'échantillons disponibles par client, permettrait probablement d'identifier un point de fonctionnement plus favorable au compromis confidentialité-performance. Sur le plan de la blockchain, une extension vers une architecture à quatre organisations Fabric distinctes renforcerait encore la décentralisation de l'infrastructure de traçabilité elle-même. Enfin, l'intégration complète du chiffrement homomorphe TenSEAL dans le pipeline d'agrégation, au-delà de la simple démonstration, et l'ajout de mécanismes de robustesse face aux contributions byzantines (par exemple l'agrégation de Krum) et de preuves de légitimité plus avancées (HMAC ou preuves à divulgation nulle de connaissance) constitueraient une extension naturelle pour renforcer encore la sécurité du système face à des participants malveillants.

Bibliographie

- [1] Canadian Institute for Cybersecurity, *CIC-IDS2018 Dataset*, University of New Brunswick. Disponible en ligne : <https://www.unb.ca/cic/datasets/ids-2018.html>
- [2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, *Communication-Efficient Learning of Deep Networks from Decentralized Data*, Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017.
- [3] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang, *Deep Learning with Differential Privacy*, Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security.
- [4] Meta AI, *Opacus – Train PyTorch models with Differential Privacy*, documentation officielle. Disponible en ligne : <https://opacus.ai>
- [5] A. Benaïssa, B. Retiat, B. Cebere, A. Belfedhal, *TenSEAL : A Library for Encrypted Tensor Operations Using Homomorphic Encryption*, arXiv preprint, 2021.
- [6] J. H. Cheon, A. Kim, M. Kim, Y. Song, *Homomorphic Encryption for Arithmetic of Approximate Numbers*, Advances in Cryptology – ASIACRYPT 2017.
- [7] Hyperledger Foundation, *Hyperledger Fabric Documentation*. Disponible en ligne : <https://hyperledger-fabric.readthedocs.io>
- [8] E. Androulaki et al., *Hyperledger Fabric : A Distributed Operating System for Permissioned Blockchains*, Proceedings of the Thirteenth EuroSys Conference, 2018.
- [9] S. Hochreiter, J. Schmidhuber, *Long Short-Term Memory*, Neural Computation, vol. 9, no. 8, 1997.
- [10] A. Paszke et al., *PyTorch : An Imperative Style, High-Performance Deep Learning Library*, Advances in Neural Information Processing Systems 32, 2019.
- [11] V. Mothukuri, R. M. Parizi, S. Pouriyeh, Y. Huang, A. Dehghantanha, G. Srivastava, *A survey on security and privacy of federated learning*, Future Generation Computer Systems, vol. 115, 2021.
- [12] S. Sicari, A. Rizzardi, A. Coen-Porisini, *Federated learning, blockchain, and IoT : a critical analysis*, IEEE Internet of Things Journal, 2021.

A. Configuration expérimentale

A.1 Environnement matériel et logiciel

TABLE A.1 – Configuration de l’environnement expérimental

Composant	Détail
Machine virtuelle	Kali Linux, 16 Go de RAM allouée
Conteneurisation	Docker, Docker Compose
Réseau blockchain	Hyperledger Fabric test-network (2 organisations, 1 orderer, 2 peers, 2 CA)
Langage chaincode	JavaScript (Node.js 18)
Framework IA	PyTorch (CPU uniquement, <code>CUDA_VISIBLE_DEVICES=""</code>)
Bibliothèques de sécurité	Opacus, TenSEAL
Langage du pipeline ML	Python 3.10+

A.2 Hyperparamètres d’entraînement

TABLE A.2 – Hyperparamètres utilisés pour l’entraînement du modèle CNN-LSTM

Hyperparamètre	Valeur
Optimiseur	Adam
Taux d’apprentissage	0,001
Taille de batch	128
Nombre d’époques locales par round	3
Nombre de rounds fédérés	5
Fonction de perte	Entropie croisée (<i>CrossEntropyLoss</i>)
Nombre de clients	4
Paramètre Dirichlet (non-IID)	$\alpha = 0,5$

A.3 Paramètres de la couche de sécurité

TABLE A.3 – Paramètres TenSEAL et Opacus utilisés

Paramètre	Valeur
Schéma de chiffrement (TenSEAL)	CKKS
Degré de modulus polynomial	8192
Tailles des coefficients	[60, 40, 40, 60]
Échelle globale	2^{40}
noise_multiplier (Opacus)	2,5
max_grad_norm (Opacus)	1,0
δ (delta)	10^{-5}

B. Détails du modèle CNN-LSTM

B.1 Définition complète de la classe du modèle

```
1 from opacus.layers import DPLSTM
2
3 class CNN_LSTM(nn.Module):
4     def __init__(self, input_size, num_classes):
5         super().__init__()
6         self.conv = nn.Sequential(
7             nn.Conv1d(1, 64, kernel_size=3, padding=1),
8             nn.ReLU(),
9             nn.MaxPool1d(2)
10        )
11        self.lstm = DPLSTM(
12            64, 128, batch_first=True, bidirectional=True
13        )
14        self.fc = nn.Sequential(
15            nn.Linear(256, 128),
16            nn.ReLU(),
17            nn.Dropout(0.3),
18            nn.Linear(128, num_classes)
19        )
20
21    def forward(self, x):
22        x = x.unsqueeze(1)
23        x = self.conv(x)
24        x = x.transpose(1, 2)
25        x, _ = self.lstm(x)
26        return self.fc(x[:, -1, :])
```

Listing B.1 – Classe CNN_LSTM complète

B.2 Partitionnement IID

```
1 def separation_iid(X, y, n_clients=4):
2     index_clients = [[] for _ in range(n_clients)]
3     for classe in np.unique(y):
4         indices = np.where(y == classe)[0]
5         np.random.shuffle(indices)
```

```

6     morceaux = np.array_split(indices, n_clients)
7     for i, morceau in enumerate(morceaux):
8         index_clients[i].extend(morceau.tolist())
9     return [(X[idx], y[idx]) for idx in index_clients]

```

Listing B.2 – Fonction de partitionnement IID entre les quatre clients

B.3 Partitionnement non-IID (Dirichlet)

```

1 def separation_non_iid(X, y, n_clients=4, alpha=0.5, seed=42):
2     np.random.seed(seed)
3     idx_clients = [[] for _ in range(n_clients)]
4     for cls in np.unique(y):
5         cls_idx = np.where(y == cls)[0]
6         np.random.shuffle(cls_idx)
7         proportions = np.random.dirichlet([alpha] * n_clients)
8         split_points = np.cumsum(
9             proportions * len(cls_idx)
10        ).astype(int)
11        split_points = np.append(0, split_points)
12        for i in range(n_clients):
13            start = split_points[i]
14            end = split_points[i + 1]
15            idx_clients[i].extend(cls_idx[start:end].tolist())
16    for i in range(n_clients):
17        np.random.shuffle(idx_clients[i])
18    return [(X[idx], y[idx]) for idx in idx_clients]

```

Listing B.3 – Fonction de partitionnement non-IID par distribution de Dirichlet

C. Extraits du chaincode et commandes Fabric

C.1 Chaincode complet (lib/flTracker.js)

```
1  'use strict';
2
3  const { Contract } = require('fabric-contract-api');
4
5  class FLTrackerContract extends Contract {
6
7      async InitLedger(ctx) {
8          console.info('Ledger FL-Tracker initialise');
9          return;
10     }
11
12     async RecordExchange(ctx, round, clientEmetteur,
13         clientDestinataire, hashPoids, modeType, timestamp) {
14         const txId = ctx.stub.getTxID();
15         const transaction = {
16             docType: 'flExchange',
17             txId: txId,
18             round: parseInt(round),
19             clientEmetteur: clientEmetteur,
20             clientDestinataire: clientDestinataire,
21             hashPoids: hashPoids,
22             modeType: modeType,
23             timestamp: timestamp,
24             submittingMSP: ctx.clientIdentity.getMSPID(),
25             submittingID: ctx.clientIdentity.getID()
26         };
27         await ctx.stub.putState(
28             txId, Buffer.from(JSON.stringify(transaction))
29         );
30         return JSON.stringify(transaction);
31     }
32
33     async GetAllTransactions(ctx) {
34         const allResults = [];
35         const iterator = await ctx.stub.getStateByRange('', '');
36         let result = await iterator.next();
37         while (!result.done) {
```

```
38     const strValue = Buffer.from(  
39         result.value.value.toString()  
40     ).toString('utf8');  
41     let record;  
42     try {  
43         record = JSON.parse(strValue);  
44     } catch (err) {  
45         record = strValue;  
46     }  
47     if (record.docType === 'flExchange') {  
48         allResults.push(record);  
49     }  
50     result = await iterator.next();  
51 }  
52 await iterator.close();  
53 return JSON.stringify(allResults);  
54 }  
55 }  
56  
57 module.exports = FLTrackerContract;
```

Listing C.1 – Chaincode fltracker – contrat complet

C.2 Commandes de déploiement du réseau Fabric

```
1 cd fabric-samples/test-network  
2 ./network.sh down  
3 ./network.sh up createChannel -c mychannel -ca  
4 ./network.sh deployCC -ccn fltracker \  
5     -ccp ../chaincode/fl-tracker/javascript -ccl javascript
```

Listing C.2 – Démarrage du réseau et déploiement du chaincode

C.3 Commande d’interrogation du registre

```
1 peer chaincode query -C mychannel -n fltracker \  
2     -c '{"function":"GetAllTransactions","Args":[]}' | jq
```

Listing C.3 – Requête de consultation de l’historique des transactions

D. Captures des résultats et courbes de convergence

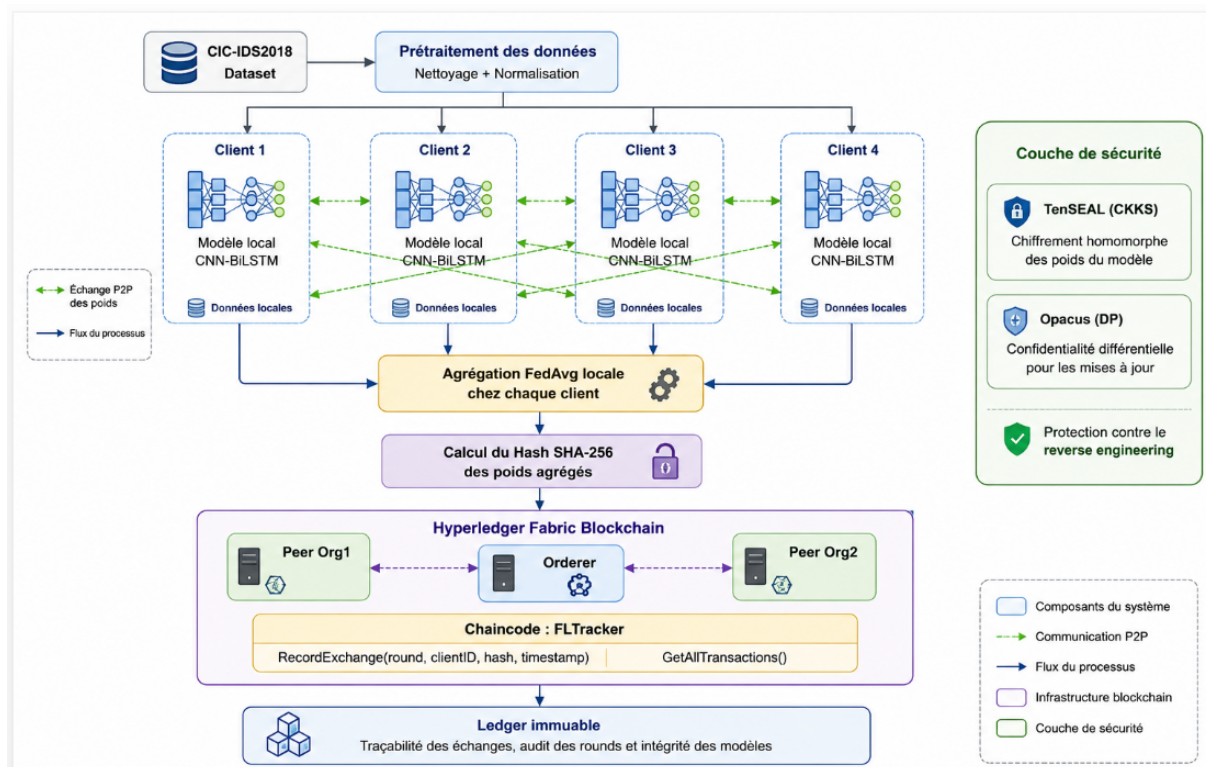


FIGURE D.1 – Architecture globale du système – apprentissage fédéré décentralisé et traçabilité blockchain

```

(venv)-(kali@kali)~-[~/fl-project]
$ docker ps
CONTAINER ID   IMAGE                                     COMMAND                                     CREATED        STATUS        PORT
db0fb04fc18d   dev-peer0.org1.example.com-fltracker_1.0-d897f5a16beb52e55220e5b90fafe8822f111ed1a40e8a61a31731c3
eecd72f5be606c8c7112c4ae248e04a0d516b25ac4daac424cebc "docker-entrypoint.s..." 30 hours ago   Up 30 hours
mple.com-fltracker_1.0-d897f5a16beb52e55220e5b90fafe8822f111ed1a40e8a61a31731c318d524fd
69cbbbc8b106   dev-peer0.org2.example.com-fltracker_1.0-d897f5a16beb52e55220e5b90fafe8822f111ed1a40e8a61a31731c31
aa72cbe75bfca1c7d0951937ca0ce4bfdd7d53fd32d6ea0de0f1b "docker-entrypoint.s..." 30 hours ago   Up 30 hours
mple.com-fltracker_1.0-d897f5a16beb52e55220e5b90fafe8822f111ed1a40e8a61a31731c318d524fd
f55f2df3a2d6   hyperledger/fabric-orderer:latest      "orderer"                                     30 hours ago   Up 30 hours   0.0
, [::]:7050→7050/tcp, 0.0.0.0:7053→7053/tcp, [::]:7053→7053/tcp, 0.0.0.0:9443→9443/tcp, [::]:9443→9443/tcp
m
d80f4a8c3a51   hyperledger/fabric-peer:latest          "peer node start"                             30 hours ago   Up 30 hours   0.0
, [::]:9051→9051/tcp, 7051/tcp, 0.0.0.0:9445→9445/tcp, [::]:9445→9445/tcp
.com
a480d635d89c   hyperledger/fabric-peer:latest          "peer node start"                             30 hours ago   Up 30 hours   0.0
, [::]:7051→7051/tcp, 0.0.0.0:9444→9444/tcp, [::]:9444→9444/tcp
.com
fcc2d77e0f4e   hyperledger/fabric-ca:latest            "sh -c 'fabric-ca-se..." 30 hours ago   Up 30 hours   0.0
, [::]:8054→8054/tcp, 7054/tcp, 0.0.0.0:18054→18054/tcp, [::]:18054→18054/tcp
4fa3df42ee8d   hyperledger/fabric-ca:latest            "sh -c 'fabric-ca-se..." 30 hours ago   Up 30 hours   0.0
, [::]:7054→7054/tcp, 0.0.0.0:17054→17054/tcp, [::]:17054→17054/tcp
1c0bcfc23a85   hyperledger/fabric-ca:latest            "sh -c 'fabric-ca-se..." 30 hours ago   Up 30 hours   0.0
, [::]:9054→9054/tcp, 7054/tcp, 0.0.0.0:19054→19054/tcp, [::]:19054→19054/tcp

```

FIGURE D.2 – Containers Docker actifs du réseau Hyperledger Fabric (sortie de `docker ps`)

```
(kali㉿kali)-[/~/fabric-project/fabric-samples/test-network]
$ peer chaincode query -C mychannel -n fltracker -c '{"function":"GetAllTransactions","Args":[]}'

[{"docType":"flExchange","txId":"0369c0ec2363d82f2d077a8391712f76e78f2180d953d32b8bd930406a2c537e",
"round":1,"clientEmetteur":"Reseau","clientDestinataire":"Client4","hashPoids":"60e24667320183f45
6516c336b263909a65b1edea4f7604ee5d974f24ea1c34f","modeType":"IID","timestamp":"2026-06-20T08:37:24.
929564","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger/OU=
admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},
{"docType":"flExchange","txId":"03ae6b213c9a7e6f094ca582be610e4ee36a2ab41b29279f5355942505ab1e70",
"round":5,"clientEmetteur":"Reseau","clientDestinataire":"Client3","hashPoids":"29c8acf70dcd605cf01
130cfce4e878e675db01e74b9e0d4c0e0a2074cc446e4","modeType":"NonIID","timestamp":"2026-06-20T08:19:0
2.936039","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger/OU
=admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},
{"docType":"flExchange","txId":"04fac5cc88752bec98d6ce2fd3554d31f890aada7538a3cb41b272dbd2c5e4ae",
"round":3,"clientEmetteur":"Reseau","clientDestinataire":"Client3","hashPoids":"ff58da7e03fd9e8f6f
26e72ff5f55444f71eeb2f2af1a9f4b9601551ce47d8d4","modeType":"NonIID","timestamp":"2026-06-20T08:13:
43.914664","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger/O
U=admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},
{"docType":"flExchange","txId":"09c599a69e807317e597577e9e6eb96179a2b31404f357940c6696f236e8fb71",
"round":1,"clientEmetteur":"Reseau","clientDestinataire":"Client2","hashPoids":"98f8335b42467bea
8a71adc3138992d7c7decfd7f1ab5a2655c57e1dad2207f","modeType":"NonIID","timestamp":"2026-06-20T11:57
:34.044842","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger
OU=admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},
{"docType":"flExchange","txId":"0b552c143d28f490c23c980f1eb4124d19a47cfc0c313c40b94951f742f4ef5b",
"round":5,"clientEmetteur":"Reseau","clientDestinataire":"Client2","hashPoids":"691df3bf228a2034
af00f22c7febe62552a9ab7bbfb165001cb05470628a3ca7","modeType":"NonIID","timestamp":"2026-06-20T09:2
1:54.790938","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger
/OU=admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},
{"docType":"flExchange","txId":"0c6ef66efc62bf48981c5c194b253c13cd31a0b88623cbcee05492e0220aaee
f","round":3,"clientEmetteur":"Reseau","clientDestinataire":"Client3","hashPoids":"adba0aa12882a1
573eb9d7fcc0f48ff98213fa0109b72d8e9e91bb58f5dedb0","modeType":"NonIID","timestamp":"2026-06-20T09:
16:08.863516","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledge
r/OU=admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.co
m"},
{"docType":"flExchange","txId":"0cf3dddeaf3154dac28b1a943e03495cd0b2eb4db3901a428f1f12da24500
a2","round":4,"clientEmetteur":"Reseau","clientDestinataire":"Client3","hashPoids":"19ed3f6c9390ae
9ba7f5bf618ecbcb55c87f7a9cd597f3887ec9f9f8c5876ed","modeType":"IID","timestamp":"2026-06-20T09:04
:30.661979","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger/
OU=admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},
{"docType":"flExchange","txId":"127ec7506ae2cc91cdccbd61bbfca7e325c2b687b48f76748a01a6714226a74",
"round":1,"clientEmetteur":"Reseau","clientDestinataire":"Client2","hashPoids":"992442f445a95309
294874f9fac50b55eff8663d0c48824f5f1aa0d0dea12af9b","modeType":"IID","timestamp":"2026-06-20T08:37:2
4.709613","submittingMSP":"Org1MSP","submittingID":"x509::/C=US/ST=North Carolina/O=Hyperledger/OU
=admin/CN=org1admin::/C=US/ST=North Carolina/L=Durham/O=org1.example.com/CN=ca.org1.example.com"},
{"docType":"flExchange","txId":"1398bda01b9ee032a7c49685dd1d22f0c43143afe5cb4356ec041c9830b42801",
```

FIGURE D.3 – Aperçu de l’historique des transactions enregistrées sur le chaincode `fltracker` (requête `GetAllTransactions`)

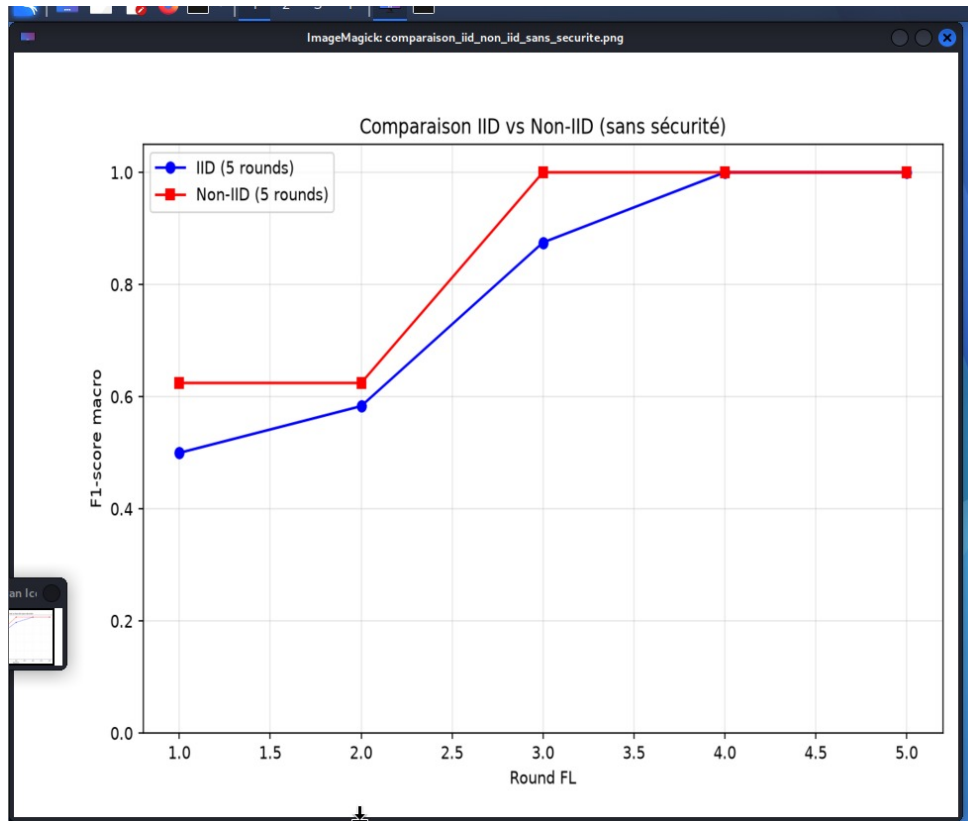


FIGURE D.4 – Convergence du F1-score en partitionnement IID et NON IID sans sécurité (script `main_5rounds.py`)

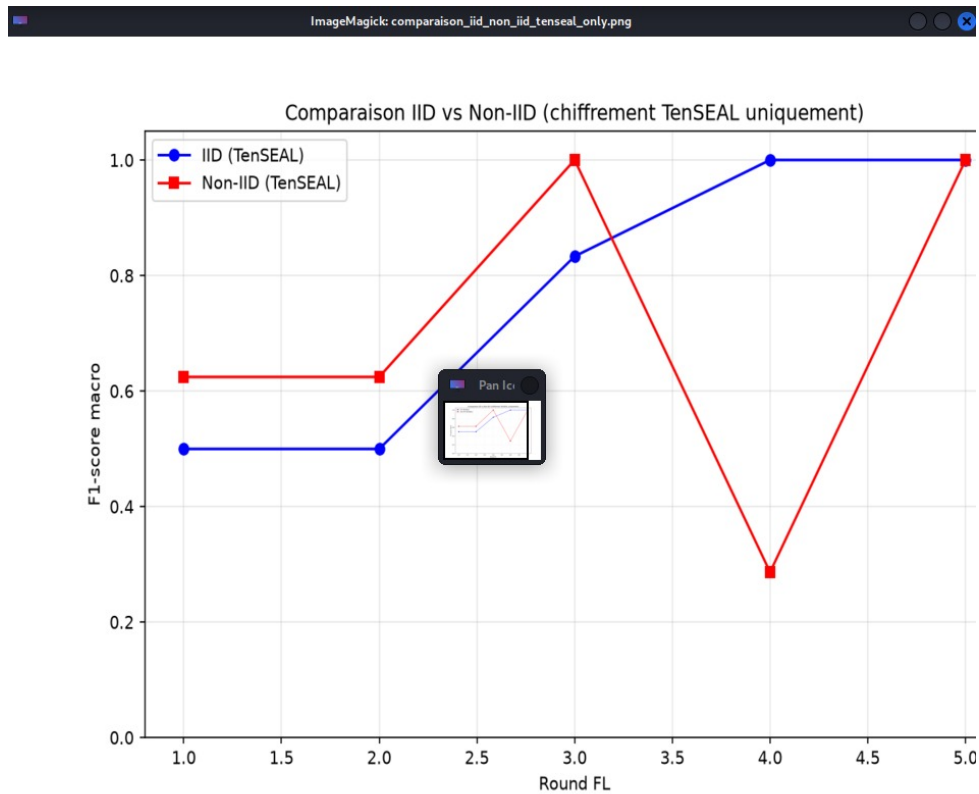


FIGURE D.5 – Convergence du F1-score en partitionnement IID et NON IID avec sécurité (TenSEAL)(script `main_tenseal_only`)

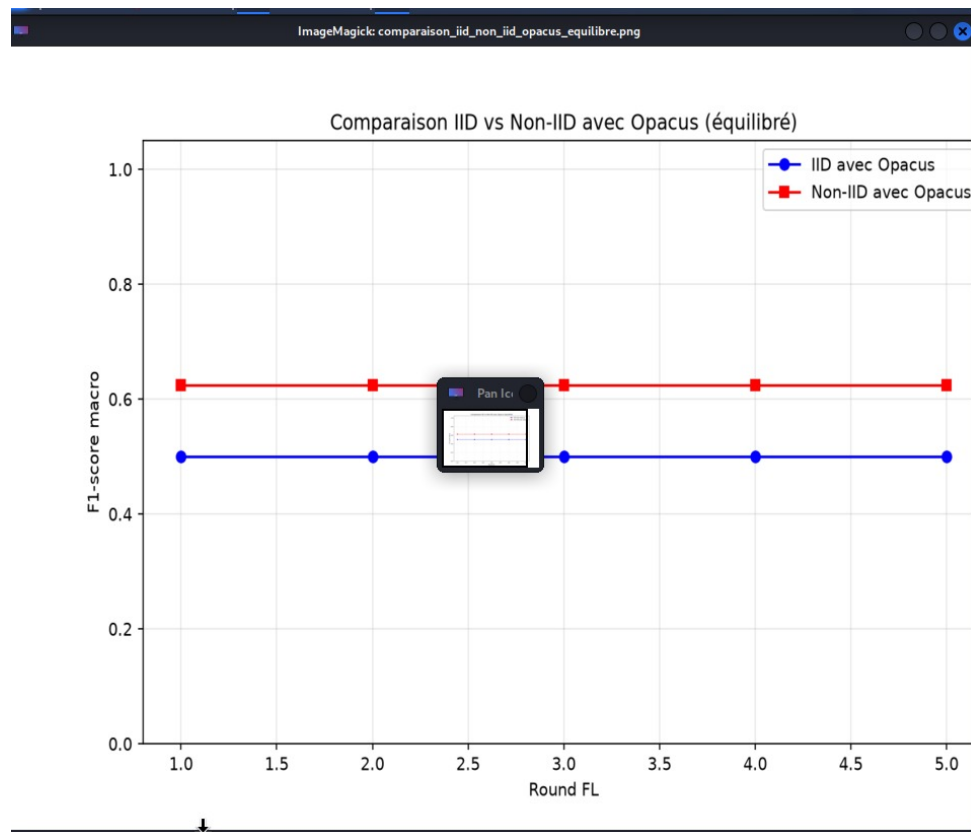


FIGURE D.6 – Comparaison des F1-scores IID vs non-IID avec Opacus activé (script `main_opacus_balanced.py`)

```

(venv)-(kali@kali)-[~/fl-project]
$ python main_opacus_balanced.py

== Chargement du dataset ==
Shape brut : (100000, 80)
Features: 77, Classes: 2, Samples: 99997

== Partitionnement IID ==
Client 1: 25000 samples, classes: [ 15 24985]
Client 2: 24999 samples, classes: [ 29 24970]
Client 3: 24999 samples, classes: [ 33 24966]
Client 4: 24999 samples, classes: [ 38 24961]

== Partitionnement Non-IID (Dirichlet  $\alpha=0.5$ ) ==
Client 1: 6850 samples, classes: [ 31 6819]
Client 2: 26342 samples, classes: [ 53 26289]
Client 3: 771 samples, classes: [ 4 767]
Client 4: 66034 samples, classes: [ 27 66007]

===== EXÉCUTION IID AVEC OPACUS (ÉQUILIBRÉ) =====

— Round 1 (IID) —
👉 Activation de Differential Privacy (Opacus)
🔒 DP  $\epsilon$  = 0.4902
🔒 DP  $\epsilon$  = 0.4902
🔒 DP  $\epsilon$  = 0.4902
🔒 DP  $\epsilon$  = 0.4902
✅ Transaction enregistrée pour Client 1
✅ Transaction enregistrée pour Client 2
✅ Transaction enregistrée pour Client 3
✅ Transaction enregistrée pour Client 4
Client 1 - F1: 0.4997 | Acc: 0.9990
Client 2 - F1: 0.4997 | Acc: 0.9990
Client 3 - F1: 0.4995 | Acc: 0.9980
Client 4 - F1: 0.4995 | Acc: 0.9980
➔ F1 moyen du round: 0.4996

— Round 2 (IID) —
👉 Activation de Differential Privacy (Opacus)
🔒 DP  $\epsilon$  = 0.4902
🔒 DP  $\epsilon$  = 0.4902
🔒 DP  $\epsilon$  = 0.4902
🔒 DP  $\epsilon$  = 0.4902
✅ Transaction enregistrée pour Client 1
✅ Transaction enregistrée pour Client 2
✅ Transaction enregistrée pour Client 3
✅ Transaction enregistrée pour Client 4
Client 1 - F1: 0.4997 | Acc: 0.9990
Client 2 - F1: 0.4997 | Acc: 0.9990
Client 3 - F1: 0.4995 | Acc: 0.9980
Client 4 - F1: 0.4995 | Acc: 0.9980
➔ F1 moyen du round: 0.4996

```

FIGURE D.7 – Extrait des journaux d'exécution du script `main_opacus_balanced.py` montrant le F1-score, le budget de confidentialité ϵ et les transactions blockchain